arXiv:2610.12341v1 [cs.AI] 8 Oct 2026

Can AI Agents Learn Their Way to the Top? Evaluating Heuristic Learning in a Long-Running Game Agent Competition

Kaisen Yang[1,*,†], Qingle Liu[1,*,†], Kejin Wang[1,*], Yicheng Zhao[1,*], Jieming Li[1,*], Shenghan Zheng[1,*], Ruize Yang[1,*], Bojun Yang[1,*], Heng Gong[1], Xiang Gao[1], Lanyue Zhang[1], Kaiyu Zhong[1], Zhuo Liu[1], Shaoxuan Li[1], Chengxi Li[1], Yong Yan[1], Weixuan Zhang[1], Tianwei Luo[1], Situ Wang[1], Youjie Zheng[1], Sihan Zhao[1], Shengyuan Wang[1,2], Huan-ang Gao[1], Jiazheng Xu[1], Xiaohui Xie[1], Wentao Han[1], Hongning Wang[1,‡]

[1] Department of Computer Science and Technology, Tsinghua University
[2] College of AI, Tsinghua University

[*] Core Contributor; [†] Project Lead; [‡] Corresponding Author.

Adversarial games have driven advances from heuristic search to reinforcement learning, yet learning and adapting strategies from limited samples remain challenging. AI agents offer an alternative by turning game experience into revisions of executable policies. Building on heuristic learning (HL), we formalize Adversarial Heuristic Learning (AHL), a paradigm that uses AI agents as learning engines to refine game policies and supporting software while keeping model weights fixed. We introduce AAArena, a benchmark comprising 12 authentic adversarial games and 1,920 archived human programs, with an evaluation protocol modeled on real-world game competitions. Agents interpret rules, choose opponents, analyze replays, and revise game agents to achieve their highest ranking within fixed match and evaluation budgets. We evaluate 7 model and harness configurations: Opus5.5 with Claude Code earns 6 gold medals (rank-1 finishes), while no evaluated configuration tops the remaining 6 human ladders. Performance is generally weaker in games with more complex rule specifications. Further experiments show that opponent selection and dense feedback support policy improvement, and that agents learn from both on-policy replays of their own matches and off-policy replays of other players' matches. These results highlight HL's potential in adversarial games and identify persistent challenges in game understanding, strategy implementation, and long-horizon policy development.

Contact: yks23@mails.tsinghua.edu.cn, lql24@mails.tsinghua.edu.cn
Project page: https://aaarena.net
Code: https://github.com/THU-CST-SAST/AAArena

1 Introduction

Adversarial games have long served as a testing ground for artificial intelligence. Early systems relied on hand-crafted heuristics, evaluation functions, and tree search; reinforcement learning and self-play subsequently advanced experience-driven policy optimization (Campbell et al., 2002; Coulom, 2007; Silver et al., 2018). Opponents form part of each player's environment, and changes in their strategies shift the learning target. Adapting and revising strategies in a changing environment with limited interaction remains a challenge (Hernandez-Leal et al., 2019).

AI agents now work in code repositories (Yang et al., 2024). They also operate graphical interfaces (Xie et al., 2024), while agent-design and recursive program-improvement methods let models revise the software supporting their behaviour (Hu et al., 2025c; Zelikman et al., 2024). These capabilities suggest that AI agents can serve as learning engines. In *Learning Beyond Gradients*, Weng (2026) describes *heuristic learning*

arXiv:2610.12341v1 [cs.AI] 8 Oct 2026

人工智能智能体能否通过学习登顶？评估长期游戏智能体竞赛中的启发式学习

Kaisen Yang[1],[*,†], Qingle Liu[1],[*,†], Kejin Wang[1],[*], Yicheng Zhao[1],[*], Jieming Li[1],[*], Shenghan Zheng[1],[*], Ruize Yang[1],[*], Bojun Yang[1],[*], Heng Gong[1], Xiang Gao[1], Lanyue Zhang[1], Kaiyu Zhong[1], Zhuo Liu[1], Shaoxuan Li[1], Chengxi Li[1], Yong Yan[1], WeixuanZhang[1], Tianwei Luo[1],Situ Wang[1], Youjie Zheng[1], Sihan Zhao[1], Shengyuan Wang[1,2], Huan-ang Gao[1], Jiazheng Xu[1],Xiaohui Xie[1],Wentao Han[1], Hongning Wang[1],[‡]

[1] 清华大学计算机科学与技术系[2] 清华大学人工智能学院

[*] 核心贡献者； [†] 项目负责人； [‡] 通讯作者。

对抗性游戏推动了从启发式搜索到强化学习的技术进步，但从有限样本中学习和调整策略仍然具有挑战性。人工智能智能体提供了另一种途径：将游戏经验转化为对可执行策略的修订。在启发式学习（HL）的基础上，我们将对抗式启发学习（AHL）形式化为一种范式：使用人工智能智能体作为学习引擎，在保持模型权重不变的同时，改进游戏策略及配套软件。我们推出了 AAArena，这一基准测试包含 12 款真实的对抗性游戏和 1,920 个归档人类程序，并采用仿照现实世界游戏竞赛设计的评估协议。智能体解读规则、选择对手、分析对局回放并修订游戏智能体，以在固定的对局和评估预算内取得尽可能高的排名。我们评估了 7 种模型与运行框架配置：Opus5.5 与 Claude Code 的组合获得了 6 枚金牌（即排名第一），而在其余 6 个人类玩家天梯中，没有任何受评估配置登顶。在规则规范更复杂的游戏中，表现通常较弱。进一步的实验表明，对手选择和密集反馈有助于策略改进，而且智能体既能从自身对局的同策略回放中学习，也能从其他玩家对局的异策略回放中学习。这些结果凸显了启发式学习在对抗性游戏中的潜力，同时揭示了游戏理解、策略实现和长时域策略开发方面持续存在的挑战。

联系方式： yks23@mails.tsinghua.edu.cn,
lql24@mails.tsinghua.edu.cn**项目主页：** https://aaarena.net**代码：**
https://github.com/THU-CST-SAST/AAArena

1 引言

对抗性游戏长期以来一直是人工智能的试验场。早期系统依赖于
人工设计的启发式规则、评估函数和树搜索；随后，强化学习和自我对弈推动了经验驱动的策略优化（Campbell 等人，2002年；Coulom，2007年；Silver 等人，2018年）。对手构成了每位玩家环境的一部分，其策略的变化会使学习目标发生偏移。在交互有限且不断变化的环境中调整和修正策略，仍然是一项挑战（Hernandez-Leal 等人，2019年）。

如今，人工智能智能体已能在代码仓库中工作（Yang 等人，2024年）。它们也能操作图形界面（Xie 等人，2024年），而智能体设计和递归程序改进方法则使模型能够修订支撑其行为的软件（Hu 等人，2025年c；Zelikman 等人，2024年）。这些能力表明，人工智能智能体可以充当学习引擎。在《超越梯度的学习》中，Weng （2026年）描述了启发式学习

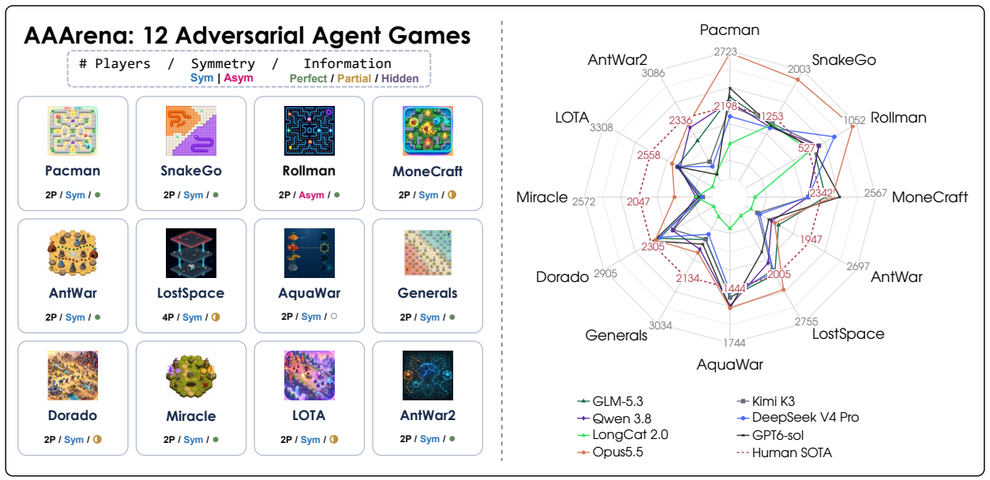

Figure 1 AAArena overview. Left: the 12 games in ascending rule-description size (abstract syntax tree node count), with player counts, role symmetry, and information structure (Table 1). Right: retained-champion Elo for seven models and human SOTA. Each spoke reports native Elo, with game-specific tick intervals; higher Elo lies farther outward. Human SOTA forms the dashed regular dodecagon, with its Elo labeled in red. Model colours match Figure 3.

(HL), in which an AI agent accumulates experience in editable software structures. The agent reads failures, logs, and replays to revise programs and their supporting software. Recent work also studies program-policy revision and tournament-based code evolution (Garnier et al., 2026; Wang et al., 2026; Fu et al., 2026; Yang et al., 2026a).

The Tsinghua University Agent Competition is an annual programming competition in which contestants build game agents to compete in adversarial games (Saiblo). Each year, organizers introduce a new game and publish its rules. Contestants interpret the rules, design executable strategies, and improve their programs through freely chosen matches to compete for leaderboard positions.

Can AI agents undertake the same process, learning from rules and game feedback to continually improve the game agents they build? This requires translating game understanding into executable policies and planning tests and revisions within a limited budget.

We formalize this feedback-driven program improvement as *Adversarial Heuristic Learning* (AHL), an instantiation of HL in adversarial tasks. The AI agent revises code policies and supporting software from game experience; the game agent executes these policies to compete. The base model's weights remain fixed.

We build *Agents for Agents Arena* (AAArena) from the competition's past games, runnable backends, and archived human programs, turning this development process into a unified evaluation task. Across 12 original games, AAArena budgets selected matches and full-pool evaluations separately to assess whether AI agents can improve their programs and climb human-program ladders.

Our strongest configuration, Opus5.5 with Claude Code, earns 6 gold medals (rank 1 in the frozen human ladders) across 12 games; no evaluated model tops the remaining 6 ladders.

Our contributions are threefold.

- **We formalize adversarial heuristic learning (AHL)**, a paradigm that uses AI agents as learning engines to learn from adversarial matches and iteratively refine executable game policies.
- **We introduce AAArena**, a benchmark comprising 12 authentic adversarial games and 1,920 archived human programs, with an evaluation protocol modeled on real-world adversarial game competitions.
- **We evaluate seven AI model–harness configurations on AAArena**, with the strongest topping six

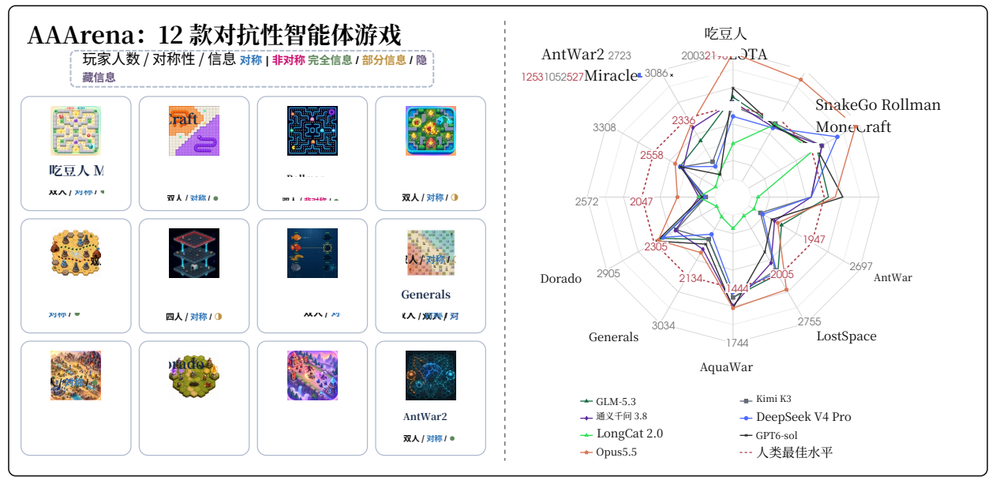

图1 AAArena概览。左：按规则描述规模（抽象语法树节点数）升序排列的12款游戏，以及各自的玩家数量、角色对称性和信息结构（表1）。右：七个模型及人类最佳水平的保留冠军的Elo评分。每条辐射轴显示原生Elo评分，刻度间隔因游戏而异；埃洛等级分越高，位置越靠外。人类最佳水平构成虚线正十二边形，其埃洛等级分以红色标注。模型颜色与图3一致。

（启发式学习），在这种学习方式中，人工智能智能体将经验积累在可编辑软件结构中。智能体通过读取失败案例、日志和对局回放来修订程序及其配套软件。近期研究还探讨了程序策略修订和基于锦标赛的代码演化（Garnier 等人，2026年；Wang 等人，2026年；Fu 等人，2026年；Yang 等人，2026a）。

清华大学智能体竞赛是一项年度编程竞赛，参赛者构建游戏智能体，在对抗性游戏中展开竞争（Saiblo）。每年，组织者都会推出一款新游戏并公布其规则。参赛者解读规则、设计可执行策略，并通过自由选择的对局改进程序，以争夺排行榜名次。

人工智能智能体能否经历同样的过程，从规则和游戏反馈中学习，持续改进自己构建的游戏智能体？这需要将游戏理解转化为可执行策略，并在有限预算内规划测试与修订。

我们将这种反馈驱动的程序改进形式化为对抗式启发式学习（AHL），即启发式学习在对抗任务中的一种实例。人工智能智能体根据游戏经验修改代码策略和配套软件；游戏智能体执行这些策略以参与竞争。基础模型的权重保持不变。

我们利用该竞赛过去的游戏、可运行后端和归档人类程序构建了*Agents for Agents Arena*（AAArena），将这一开发过程转化为统一的评测任务。在 12 款原创游戏中，AAArena 分别为选定对局和全池评估设置预算，以评估人工智能智能体能否改进其程序，并提升在人类程序天梯中的排名。

我们最强的配置是 Opus5.5 搭配 Claude Code，在 12 款游戏中赢得了 6 枚金牌（在冻结的人类排行榜中排名第 1）；在其余 6 个排行榜中，没有任何受评模型登顶。

我们的贡献有三个方面。

- **我们将对抗式启发式学习（AHL）形式化**，这是一种以人工智能智能体作为学习引擎，从对抗比赛中学习并迭代优化可执行游戏策略的范式。
- **我们推出 AAArena**，这是一个包含 12 款真实对抗性游戏和 1,920 个归档人类程序的基准测试，其评估协议以现实世界的对抗游戏竞赛为蓝本。
- **我们评估了七　　在 AAArena 上的 n 种人工智能模型与运行框架配置**，其中最强的配置登顶了 si　　x

human ladders, while the remaining six games generally have more complex rules and remain untopped by any evaluated configuration. Further experiments show the importance of opponent selection and dense feedback, and the potential of off-policy experience from other players' replays to overcome learning plateaus. Case studies reveal persistent challenges in game understanding, strategy implementation, and long-horizon policy development.

2 Adversarial Heuristic Learning

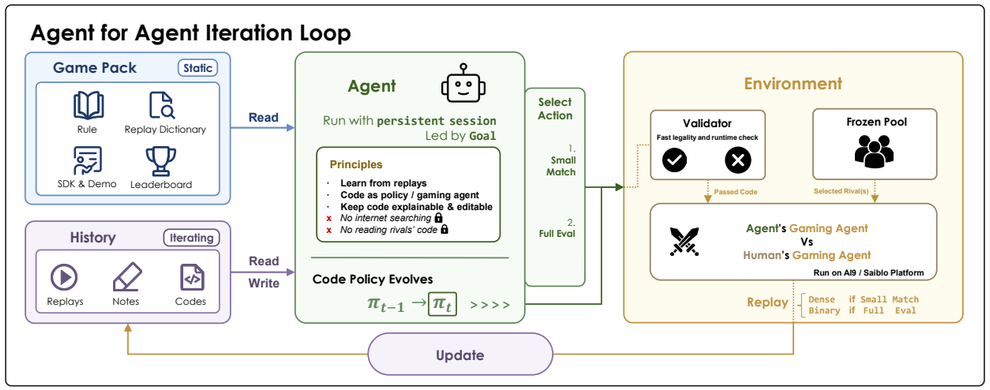

Figure 2 Agent for agent iteration loop. An AI agent uses game resources, replays, and retained notes to revise an executable game-agent policy. Validated snapshots enter small matches against selected opponents or full-pool evaluations against the frozen pool. Small matches return dense replays; full-pool evaluations return outcomes, Elo, and rank. The agent retains development history across revisions.

2.1 Preliminaries

Game agent. An adversarial game $\mathcal{G}$ has a shared environment and $n \geq 2$ game agents, with conflicting objectives for at least two agents. Games can assign asymmetric roles and restrict each agent's observations. Each game agent i executes a *code policy* P_i that selects actions as

$$a_t^i \sim \pi_{P_i}(\cdot \mid h_t^i),$$

where h_t^i is its permitted observation history and a_t^i is its action. The environment processes these actions to determine the outcome, while the submitted program acts autonomously throughout the match.

AI agent. An AI agent $\mathcal{A}_\theta$ develops the game agent's policy between matches; θ denotes the base model's parameters. It reads game resources, source code, and feedback to revise the policy and its supporting software. In *Agents for Agents*, the resulting game agent's competitive performance measures the AI agent's policy-development ability.

Heuristic learning. *Heuristic learning* (HL) incorporates experience through an AI agent's direct revisions to a software system (Weng, 2026). Feedback may include rewards, tests, logs, or replays. The agent can revise policy logic, state representations, tests, configuration, and memory. We call the executable policy and its supporting software and records the *heuristic system*.

2.2 Heuristic learning in adversarial games

Definition 1 (Adversarial heuristic learning)**.** *Adversarial heuristic learning (AHL) is an instantiation of HL in adversarial tasks. An AI agent uses experience and feedback to revise a heuristic system and improve competitive performance, while the base model's parameters θ remain fixed.*

人类玩家天梯，而其余六款游戏的规则通常更复杂，尚无任何受评估的配置登顶。进一步的实验表明，对手选择和密集反馈十分重要，利用其他玩家对局回放中的离策略经验有望突破学习平台期。案例研究揭示了游戏理解、策略实现和长时域策略开发方面持续存在的挑战。

2 对抗式启发式学习

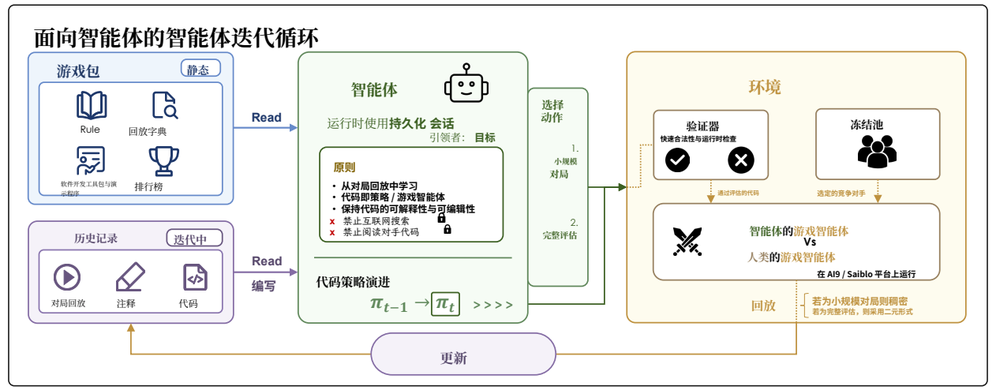

图 2 面向智能体的智能体迭代循环。人工智能智能体利用游戏资源、对局回放和留存笔记来修订可执行游戏智能体策略。已验证快照会参与针对选定对手的小规模对局，或参与针对冻结池的全池评估。小规模对局返回密集回放；全池评估返回结果、埃洛等级分和排名。智能体在历次修订中保留开发历史。

2.1 预备知识

游戏智能体。对抗性游戏 $\mathcal{G}$ 具有一个共享环境和 $n \geq 2$ 个游戏智能体，其中至少两个智能体具有冲突目标。游戏可以分配非对称角色，并限制每个智能体的观测。每个游戏智能体 i 执行一个代码策略 P_i，按如下方式选择动作

$$a_t^i \sim \pi_{P_i}(\cdot \mid h_t^i),$$

其中，h_t^i 为其获准获取的观测历史，a_t^i 为其动作。环境处理这些动作以确定结果，而提交的程序在整场对局中自主运行。

人工智能智能体。人工智能智能体 $\mathcal{A}_\theta$ 在对局之间开发游戏智能体的策略；θ 表示基础模型的参数。它读取游戏资源、源代码和反馈，以修订策略及其配套软件。在*Agents for Agents*中，所生成的游戏智能体的竞技表现用于衡量人工智能智能体的策略开发能力。

启发式学习。启发式学习（启发式学习）通过人工智能智能体对软件系统的直接修订来融入经验（Weng，2026年）。反馈可以包括奖励、测试、日志或对局回放。智能体可以修订策略逻辑、状态表示、测试、配置和记忆。我们将可执行策略及其配套软件和记录称为启发式系统。

2.2 对抗性游戏中的启发式学习

定义 1（对抗式启发式学习）。对抗式启发学习（*AHL*）是启发式学习在对抗任务中的一种实例。人工智能智能体利用经验和反馈来修订启发式系统并提升竞技表现，而基础模型的参数 θ 保持不变。

The heuristic system consists of an executable policy P and development state C, which stores supporting tools, tests, experiment records, and memory. Figure 2 summarizes the interaction loop; Algorithm 1 gives its formal form. The AI agent acquires game experience, interprets feedback, and revises the heuristic system. It chooses how to collect experience and organize revisions; the algorithm leaves feedback sources, evaluation schedules, and stopping criteria to the task setting.

Algorithm 1 Adversarial Heuristic Learning (AHL)

Input: AI agent $\mathcal{A}_\theta$ with fixed parameters θ; game resources R;
initial policy P_0 and development state C_0
$(P, C) \leftarrow (P_0, C_0)$
while the stopping criterion remains unmet **do**
  Acquire game experience through interactions or available records
  Interpret feedback using $\mathcal{A}_\theta$ and (R, P, C)
  Revise P and its supporting state C from the feedback
end while
Output: executable policy P

3 AAArena

3.1 Games and opponent pools

The 12 competition games cover tactical combat, territory control, resource management, and survival. Each has a verified, frozen opponent pool (Table 1). Agents receive rules, SDKs, replay documentation, a leaderboard, and a permitted reference program. Appendices A and B describe the games and resources.

Rule-description size. Kolmogorov complexity $K(R) = \min_{p:U(p)=R} |p|$ is the length of the shortest program describing R on a fixed universal machine U (Vitányi, 2020). We report $C_{\mathrm{AST}}(G) = |\mathrm{AST}(R_G)|$, the node count of the rules' abstract syntax tree (AST), and rule atoms (RA), the count of independently changeable rule propositions. These representation-dependent counts describe rule specifications; they do not estimate exact Kolmogorov complexity or optimal-play difficulty. The same rule-analysis procedure gives *Go* 501 AST nodes and 125 rules, and *chess* 1,551 AST nodes and 238 rules. The suite spans 1,127–6,368 AST nodes and 219–1,008 rules.

Game	Strategic setting	Pool programs	AST nodes	Rules (RA)
Pacman	Competitive maze collection	44	1,127	219
SnakeGo	Snake movement and territory	141	1,538	308
Rollman	Asymmetric maze pursuit	64	1,539	312
MoneCraft	Mining and resource control	112	1,823	347
AntWar	Tower defence and economy	114	2,160	368
LostSpace	Multiplayer survival and escape	111	2,297	426
AquaWar	Hidden-identity tactical combat	170	2,531	439
Generals	Territorial and army control	197	2,683	461
Dorado	Resource competition and army development	323	4,241	616
Miracle	Hex-grid unit tactics	253	4,391	581
LOTA	Hero control and lane combat	200	4,513	684
AntWar2	Ordered tower-defence operations	191	6,368	1,008

Table 1 The 12 games, ordered by increasing abstract syntax tree (AST) node count for their rule representations. RA counts rule atoms, or independently changeable rule propositions. Pool sizes count executable programs, not unique human participants.

启发式系统由可执行策略 P 和开发状态 C 组成，后者存储辅助工具、测试、实验记录和记忆。图 2 概述了交互循环；算法 1 给出了其形式化描述。人工智能智能体获取游戏经验、解读反馈并修订启发式系统。它自行选择如何收集经验和组织修订；算法将反馈来源、评估安排和停止准则留给具体任务设定。

算法 1 对抗式启发学习（AHL）

输入： 具有固定参数 θ 的人工智能智能体 $\mathcal{A}_\theta$；游戏资源 R；
初始策略 P_0 和开发状态 C_0
$(P, C) \leftarrow (P_0, C_0)$
当停止条件尚未满足时**执行**
  通过交互或现有记录获取游戏经验
  使用 $\mathcal{A}_\theta$ 和 (R, P, C) 解读反馈
  根据反馈修订 P 及其支撑状态 C
结束循环
输出： 可执行策略 P

3 AAArena

3.1 游戏与对手池

这 12 款竞赛游戏涵盖战术战斗、领地控制、资源管理和生存。每款游戏都有一个经过验证且已冻结的对手池（表 1）。智能体会获得规则、软件开发工具包、回放文档、排行榜以及一个获准使用的参考程序。附录 A 和 B 介绍了这些游戏及相关资源。

规则描述规模。柯尔莫哥洛夫复杂度 $K(R) = \min_{p:U(p)=R} |p|$ 是描述 R 的最短程序在固定通用机上的长度 U（Vitányi，2020年）。我们报告 $C_{\mathrm{AST}}(G) = |\mathrm{AST}(R_G)|$，即规则的抽象语法树（AST）的节点数，以及规则原子（RA），即可独立更改的规则命题的数量。这些依赖于表示方式的计数描述的是规则规范；它们并非对精确柯尔莫哥洛夫复杂度或最优对弈难度的估计。采用同一规则分析流程，围棋得到501个抽象语法树节点和125条规则，国际象棋得到1,551个抽象语法树节点和238条规则。该套件涵盖的范围为1,127–6,368个抽象语法树节点和219–1,008条规则。

Game	策略环境	Pool 程序	AST 节点	规则 (RA)
吃豆人	竞争性迷宫收集	44	1,127	219
SnakeGo	蛇形移动与领地	141	1,538	308
Rollman	非对称迷宫追逐	64	1,539	312
MoneCraft	采矿与资源控制	112	1,823	347
AntWar	塔防与经济	114	2,160	368
LostSpace	多人生存与逃脱	111	2,297	426
AquaWar	隐藏身份战术战斗	170	2,531	439
Generals	领土与军队控制	197	2,683	461
Dorado	资源争夺与军队发展	323	4,241	616
Miracle	六边形网格单位战术	253	4,391	581
LOTA	英雄控制与兵线战斗	200	4,513	684
AntWar2	有序塔防操作	191	6,368	1,008

表 1 这 12 个游戏按其规则表示的抽象语法树（AST）节点数递增排序。规则原子统计规则原子的数量，即可独立更改的规则命题的数量。池大小统计的是可执行程序的数量，而非去重后的人类参与者数量。

3.2 Interaction interfaces

Small matches support targeted diagnosis by letting the AI agent select opponents and inspect detailed replays. **Full-pool evaluations** measure competitive progress against the complete opponent pool and return Elo, rank, and per-opponent outcomes. The first channel concentrates feedback on specific strategic hypotheses; the second supplies a common reference for comparing snapshots. Both evaluate immutable programs through the same game referee and return feedback for subsequent revisions.

3.3 Evaluation protocol

Each run receives two budgets: B_s small-match units (one per requested opponent) and B_ℓ full-pool evaluations. Default limits are $B_s = 128$ and $B_\ell = 16$. Within these limits, the agent chooses when to revise, validate locally, or request either kind of feedback.

The final result is the highest officially evaluated Elo attained during the run, together with its corresponding rank. If $\mathcal{V}$ is the set of officially evaluated snapshots, including the common initialization baseline, then

$$\mathrm{Score}(B_s, B_\ell) = \max_{P \in \mathcal{V}} \widehat{r}(P).$$

We retain the policy attaining this score, allowing later revisions to explore without discarding a stronger snapshot. Opponent Elo ratings stay fixed throughout the run. Appendix B specifies ranking, budget accounting, and the baseline's separate allowance.

4 Experiments

4.1 Performance of AI Agents Developing Game Agents

Table 2 Main results across 12 adversarial games, ordered by rule complexity. Rule complexity uses AST node counts (Section 3.1). A green check marks at least one configuration reaching rank 1; a red cross marks none. Each model entry gives Elo (↑) and rank (↓) under a 128-small/16-full budget; a gold medal marks rank 1. Bold marks the best value per row. All models use `max` reasoning effort. Opus5.5 uses Claude Code; others use Codex.

Rule complexity: low (top) to high (bottom) ↓

Game	Rule complexity	Topped	Opus5.5		GPT6-sol		GLM-5.3		Kimi K3		DeepSeek V4 Pro		Qwen 3.8		LongCat 2.0	
			Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank
Pacman	1,127	✓	**2703.9**	**1**	2367.6	1	2281.6	1	2196.3	2	2096.8	4	2236.1	1	1835.0	10
SnakeGo	1,538	✓	**1860.9**	**1**	1129.6	4	1141.2	4	1165.9	3	1097.0	5	1107.5	5	1153.3	4
Rollman	1,539	✓	**1010.2**	**1**	581.2	1	581.2	1	636.9	1	807.7	1	636.9	1	534.2	1
MoneCraft	1,823	✓	2387.1	1	**2418.2**	**1**	2359.2	1	2288.2	3	2288.2	3	2288.2	3	2069.9	9
AntWar	2,160	×	1405.1	5	1311.7	6	**1464.2**	**5**	1130.2	9	1167.2	9	1355.0	5	1028.3	11
LostSpace	2,297	✓	**2220.2**	**1**	1625.2	7	1980.1	2	1916.1	2	1956.8	2	1797.7	3	1052.9	77
AquaWar	2,531	✓	**1552.5**	**1**	**1552.5**	**1**	1494.1	1	1494.1	1	1464.5	1	1539.7	1	1116.1	33
Generals	2,683	×	**1689.6**	**10**	1528.4	13	1441.2	17	1433.3	17	1340.8	23	1624.9	10	1010.3	58
Dorado	4,241	×	2252.4	2	**2271.5**	**2**	2210.3	2	2023.4	16	2187.8	3	2035.8	14	1512.4	156
Miracle	4,391	×	**1704.8**	**6**	1469.5	20	1436.8	26	1432.8	26	1448.8	24	1500.2	18	1500.2	18
LOTA	4,513	×	**2224.2**	**15**	2140.3	22	2147.4	22	2133.2	22	2140.3	22	2099.0	25	1585.1	74
AntWar2	6,368	×	**2234.4**	**5**	1440.3	34	1977.1	9	1644.2	19	1569.6	25	2183.5	6	1460.0	31
Gold medals ↑				**6**		4		4		2		2		3		1

Setup. We evaluate seven model–harness configurations with the default 128-small/16-full budgets and retained-champion scoring from Section 3.3. All models use `max` reasoning effort. Opus5.5 uses Claude Code; the other models use Codex, so the comparison includes both model and harness differences. Runs stop upon reaching rank 1. We repeat each setting three times and report the median result; each run contributes its best officially evaluated policy. Main runs use *on-policy* replays from matches involving the learner's own

3.2 交互接口

小规模对局允许人工智能智能体选择对手并查看详细的对局回放，从而支持针对性诊断。**全池评估**衡量相对于完整对手池的竞争进展，并返回埃洛等级分、排名和各对手对战结果。前一种渠道将反馈集中于特定的策略假设；后一种渠道则为比较快照提供共同参照。两者均通过同一游戏裁判评估不可变程序，并返回反馈以供后续修订。

3.3 评估协议

每次运行获得两项预算： B_s 个小规模对局单元（每请求一个对手消耗一个单元）和 B_ℓ 次全对手池评估。默认上限为 $B_s = 128$ 和 $B_\ell = 16$。在这些限制内，智能体自行选择何时修订、进行本地验证或请求任一种反馈。

最终结果为运行期间经官方评估获得的最高埃洛等级分及其对应排名。若 $\mathcal{V}$ 为经官方评估的快照集合，其中包括共同的初始化基线，则

$$\mathrm{Score}(B_s, B_\ell) = \max_{P \in \mathcal{V}} \widehat{r}(P).$$

我们保留达到该分数的策略，使后续修订能够在不丢弃更强快照的情况下进行探索。对手的Elo评分在整个运行过程中保持不变。附录B 规定了排名、预算核算以及基线的单独配额。

4 实验

4.1 人工智能智能体开发游戏智能体的表现

表2 按规则复杂度排序的12种对抗性游戏的主要结果。规则复杂度采用抽象语法树节点数衡量（第3.1节）。绿色对勾表示至少有一种配置达到排名第1；红色叉号表示没有任何配置达到。各模型条目给出在128次小规模测试/16次完整测试预算下的埃洛等级分（↑）和排名（↓）；金牌表示排名第1。粗体表示每行的最佳值。所有模型均采用最高推理强度。Opus5.5使用Claude Code；其他模型使用Codex。

规则复杂度：从低（顶部）到高（底部） ↓

Game	Rule 复杂度	是否登顶	Opus5.5		GPT6-sol		GLM-5.3		Kimi K3		DeepSeek V4 Pro		通义千问 3.8		LongCat 2.0	
			埃洛	排名	埃洛	排名	埃洛	排名	埃洛	排名	埃洛	排名	埃洛	排名	埃洛	排名
吃豆人	1,127	✓	**2703.9**	**1**	2367.6	1	2281.6	1	2196.3	2	2096.8	4	2236.1	1	1835.0	10
SnakeGo	1,538	✓	**1860.9**	**1**	1129.6	4	1141.2	4	1165.9	3	1097.0	5	1107.5	5	1153.3	4
Rollman	1,539	✓	**1010.2**	**1**	581.2	1	581.2	1	636.9	1	807.7	1	636.9	1	534.2	1
MoneCraft	1,823	✓	2387.1	1	**2418.2**	**1**	2359.2	1	2288.2	3	2288.2	3	2288.2	3	2069.9	9
AntWar	2,160	×	1405.1	5	1311.7	6	**1464.2**	**5**	1130.2	9	1167.2	9	1355.0	5	1028.3	11
LostSpace	2,297	✓	**2220.2**	**1**	1625.2	7	1980.1	2	1916.1	2	1956.8	2	1797.7	3	1052.9	77
AquaWar	2,531	✓	**1552.5**	**1**	**1552.5**	**1**	1494.1	1	1494.1	1	1464.5	1	1539.7	1	1116.1	33
Generals	2,683	×	**1689.6**	**10**	1528.4	13	1441.2	17	1433.3	17	1340.8	23	1624.9	10	1010.3	58
Dorado	4,241	×	2252.4	2	**2271.5**	**2**	2210.3	2	2023.4	16	2187.8	3	2035.8	14	1512.4	156
Miracle	4,391	×	**1704.8**	**6**	1469.5	20	1436.8	26	1432.8	26	1448.8	24	1500.2	18	1500.2	18
LOTA	4,513	×	**2224.2**	**15**	2140.3	22	2147.4	22	2133.2	22	2140.3	22	2099.0	25	1585.1	74
AntWar2	6,368	×	**2234.4**	**5**	1440.3	34	1977.1	9	1644.2	19	1569.6	25	2183.5	6	1460.0	31
金牌 ↑				**6**		4		4		2		2		3		1

初始设置。我们使用第 3.3 节中的默认 128 次小规模/16 次完整规模预算和保留冠军评分，对七种模型–运行框架配置进行评估。所有模型均使用 `max` 推理强度。Opus5.5 使用 Claude Code；其他模型使用 Codex，因此比较既涉及模型差异，也涉及运行框架差异。运行在达到排名第 1 时停止。我们对每种设置重复运行三次，并报告结果的中位数；每次运行均提供其经过正式评估的最佳策略。主要运行使用同策略对局回放，这些回放来自学习者自身所用

policy.[1] Appendix B.1 describes the implementation. We use GLM-5.3 for subsequent analyses because its complete interaction logs and fixed harness support comparisons across runs.

Results. AI-developed game agents top six frozen human ladders; every evaluated configuration falls short of rank 1 on the other six (Table 2 and Figures 1 and 3). Opus5.5 with Claude Code earns 6 gold medals and the highest or tied-highest Elo on nine games. Several configurations reach rank 1 on *Rollman* and *AquaWar*, whereas only Opus5.5 with Claude Code does so on *LostSpace* and *SnakeGo*.

The remaining gaps also differ across games. The best policy ranks second on *Dorado*, while the best policies on *LOTA*, *Generals*, and *Miracle* remain outside the top five. Gold medals summarize rank-1 coverage; per-game Elo and rank preserve these differences without averaging across game-specific rating scales.

Five of the six games with smaller rule descriptions have a rank-1 finish, compared with one of the six games with larger descriptions (Table 2). This pattern links larger rule specifications with gaps in competitive policy development.

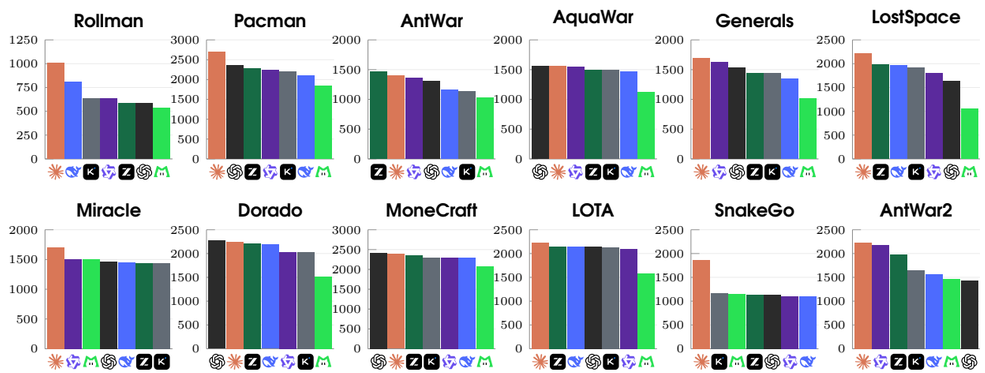

Figure 3 Per-game Elo of the retained champion. Each game orders model configurations by Elo.

Takeaway. The strongest configuration surpasses human champions in six games, while challenging games remain beyond the evaluated agents. Performance is generally weaker in games with more complex rule specifications, highlighting the challenge of translating rule understanding into competitive policies.

4.2 How Do Policies Evolve During Learning?

4.2.1 Evolution Patterns Across Games

We examine the 12 GLM-5.3 runs under the default 128-small/16-full budget to distinguish how policies improve across games. Each trajectory records the Elo and rank of individual full-pool submissions, the order of small-match and full-pool requests, and token use. Submission scores can fall after a revision even though the retained-champion score keeps the best result. The trajectories follow three patterns:

Success: reaching rank 1. *Rollman*, *MoneCraft*, *Pacman*, and *AquaWar* reach rank 1 and stop before exhausting the full-pool evaluation budget. These runs show that feedback-driven program revision can produce policies that top the human-program ladders within the given interaction limits.

Progress: gains with regressions. *AntWar*, *Dorado*, *LostSpace*, *SnakeGo*, and *AntWar2* improve overall, but individual revisions can reduce performance. *Dorado* makes several consecutive gains in the middle of its run, whereas *AntWar2* suffers a sharp intermediate drop and *LostSpace* ends with prolonged fluctuations. Retaining the best evaluated policy preserves earlier gains when later edits regress.

[1] Here, on-policy and off-policy label replay sources, not reinforcement-learning algorithm classes.

策略参与的对局。[1] 附录 B.1 介绍了实现细节。我们使用 GLM-5.3 进行后续分析，因为其完整的交互日志和固定测试框架支持跨运行比较。

结果。人工智能开发的游戏智能体在六个冻结的人类排行榜中登顶；在其余六个排行榜中，所有受评配置均未能排名第一（表 2 及图1 和3）。搭配 Claude Code 的 Opus5.5 获得了 6 枚金牌，并在九款游戏中取得最高或并列最高的埃洛等级分。多种配置在*Rollman* 和*AquaWar*中排名第一，而在*LostSpace* 和*SnakeGo*中，只有搭配 Claude Code 的 Opus5.5 做到了这一点。

各游戏中仍存的差距也有所不同。最佳策略在 *Dorado* 中排名第二，而在 *LOTA*、*Generals* 和 *Miracle* 中的最佳策略仍未进入前五。金牌数量概括了榜首覆盖率；各游戏的埃洛等级分和排名则保留了这些差异，避免了对游戏专属评分量表进行跨游戏平均。

在规则描述较短的六款游戏中，有五款取得了排名第一的成绩；相比之下，在规则描述较长的六款游戏中，只有一款取得了这一成绩（表 2）。这一模式表明，篇幅更长的规则规范与竞争性策略开发中的差距存在关联。

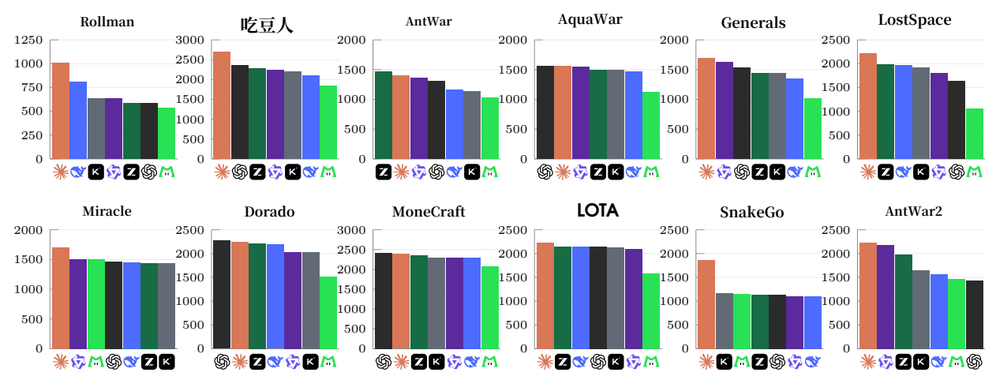

图 3 保留的冠军在各游戏中的埃洛等级分。每款游戏中的模型配置均按埃洛等级分排序。

要点。最强配置在六款游戏中超越了人类冠军，但在具有挑战性的游戏中，受评估智能体仍未能达到这一水平。对于规则规范更复杂的游戏，表现通常较弱，凸显了将规则理解转化为竞争性策略的挑战。

4.2 策略在学习过程中如何演化?

4.2.1 不同游戏中的演化模式

我们考察了默认预算（128 次小规模对局／16 次全对手池评估）下的 12 次 GLM-5.3 运行，以区分不同游戏中策略的改进方式。每条轨迹都记录了各次全池提交的埃洛等级分和排名、小规模对局请求与全池请求的先后顺序，以及词元使用量。修订后的提交得分可能下降，但保留冠军得分始终保留最佳结果。这些轨迹呈现出三种模式：

成功：达到排名第 1。*Rollman*、*MoneCraft*、吃豆人和*AquaWar* 均达到排名第 1，并在耗尽全池评估预算之前停止。这些运行表明，反馈驱动的程序修订能够在给定的交互限制内产生登顶人类程序天梯的策略。

进展：提升伴随性能回退。*AntWar*、*Dorado*、*LostSpace*、*SnakeGo*和*AntWar2* 的表现总体有所改善，但个别修订可能导致性能下降。*Dorado* 在运行中期连续取得多次提升，而*AntWar2* 在中途出现性能骤降，*LostSpace* 则在末期持续波动。当后续修改导致性能回退时，保留最佳的已评估策略可维持先前取得的提升。

[1] 此处，同策略和异策略标记的是回放来源，而非强化学习算法类别。

Stagnation: persistent plateaus. *Generals*, *Miracle*, and *LOTA* show extended intervals with little improvement after initial adaptation. Further revisions and evaluations repeatedly return to similar performance levels, showing that continued interaction does not consistently yield stronger policies within these runs.

Appendix B.6 presents the complete trajectories in Figures 11–13. We next trace the code changes behind *Dorado*'s progress to show how match feedback leads to specific policy revisions.

4.2.2 Case Study: Dorado

GLM-5.3 improves its *Dorado* policy from rank 41 to rank 2. Figure 4 traces its performance, and Table 3 shows the corresponding code changes.

Dorado has two players, each controlling a base and recruiting heroes from four classes. Mining supplies gold for recruitment and for upgrading heroes near the base. Destroying the opposing base wins; simultaneous destruction draws. If both bases remain at the turn limit, gold plus the level-based value of each player's heroes determines the result.

Correct mine allocation. The central mine stays open, while other mines have limited opening windows. Each mine uses the highest individual mining rate; sending several heroes to one mine does not add their yields. The agent excludes dead miners from occupancy counts, penalizes assigning several miners to the same mine, and tracks mine opening windows. After these revisions, the policy reaches rank 14.

Concentrate upgrades. The agent then upgrades two core heroes while preserving mining income. The next full-pool evaluation raises the policy from rank 14 to rank 3 (+147.7 Elo), the largest adjacent gain.

Refine counterattacks. The agent later allows a one-hero counterattack only after sustained nearby aggression and reaches rank 2. Other attempts, including a four-hero roster and unconditional use of the `Sacrifice` ability, regress to ranks 46 and 43. The agent restores its best policy before trying another revision.

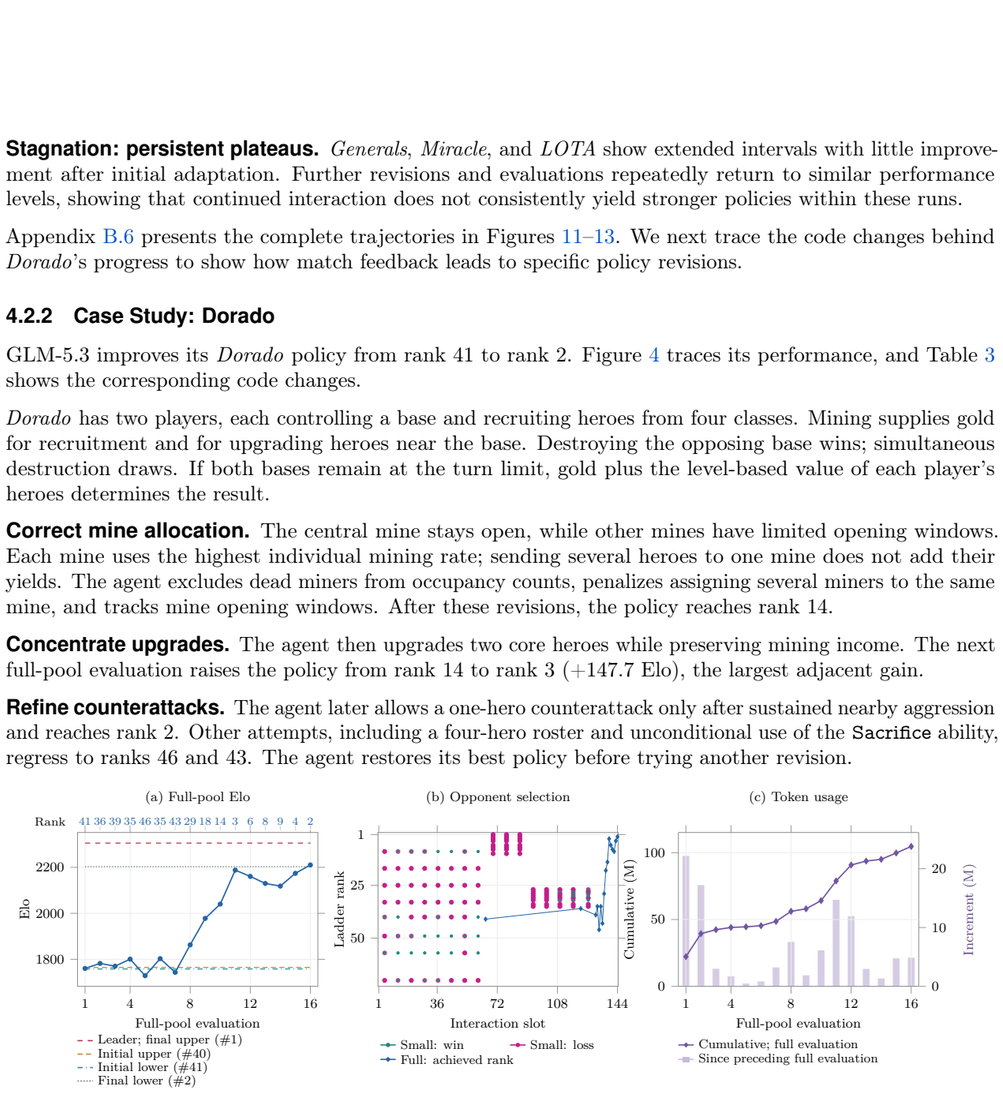

Figure 4 GLM-5.3 on *Dorado*. Full-pool Elo and rank, small–full interaction order, and token use under the 128/16 budget. Appendix B.5 specifies budget-slot placement, batch markers, rank annotations, and token accounting.

停滞：持续平台期。*Generals*、*Miracle*和*LOTA* 在初始适应后，均出现了长时间几乎没有改善的情况。进一步的修订和评估反复得到相近的性能水平，表明在这些运行过程中，持续交互并不总能产生更强的策略。

附录 B.6在图 11–13 中展示了完整轨迹。接下来，我们追溯促成*Dorado*进展的代码变更，展示对局反馈如何带来具体的策略修订。

4.2.2 案例研究：Dorado

GLM-5.3 将其 *Dorado* 策略的排名从第 41 名提升至第 2 名。图 4 展示了其性能变化，表 3展示了相应的代码变更。

Dorado 有两名玩家，每名玩家控制一座基地，并从四种职业中招募英雄。采矿提供金币，用于招募英雄以及为基地附近的英雄升级。摧毁对方基地即可获胜；双方基地同时被摧毁则为平局。如果达到回合上限时双方基地均仍存在，则根据每名玩家的金币与其英雄基于等级的价值之和判定结果。

纠正矿点分配。中央矿点始终开放，而其他地雷仅在有限的开放时段内可用。每个矿点采用最高的单体采矿速率；派遣多名英雄前往同一矿点并不会叠加其产量。该智能体在占用计数中排除已死亡矿工，对将多名矿工分配至同一矿点施加惩罚，并跟踪矿场开放时间窗口。经过这些修正，该策略的排名升至第 14 名。

集中升级。随后，该智能体在维持采矿收入的同时，集中升级两名核心英雄。下一次全对手池评估中，该策略从第 14 名升至第 3 名（+147.7 埃洛等级分），这是相邻两次评估之间的最大提升。

改进反击。随后，智能体仅在附近持续出现进攻行为后才允许单英雄反击，并取得第 2 名。其他尝试，包括采用四英雄阵容和无条件使用牺牲技能，均导致排名下滑，分别降至第 46 名和第 43 名。智能体在尝试下一次修订前恢复了其最佳策略。

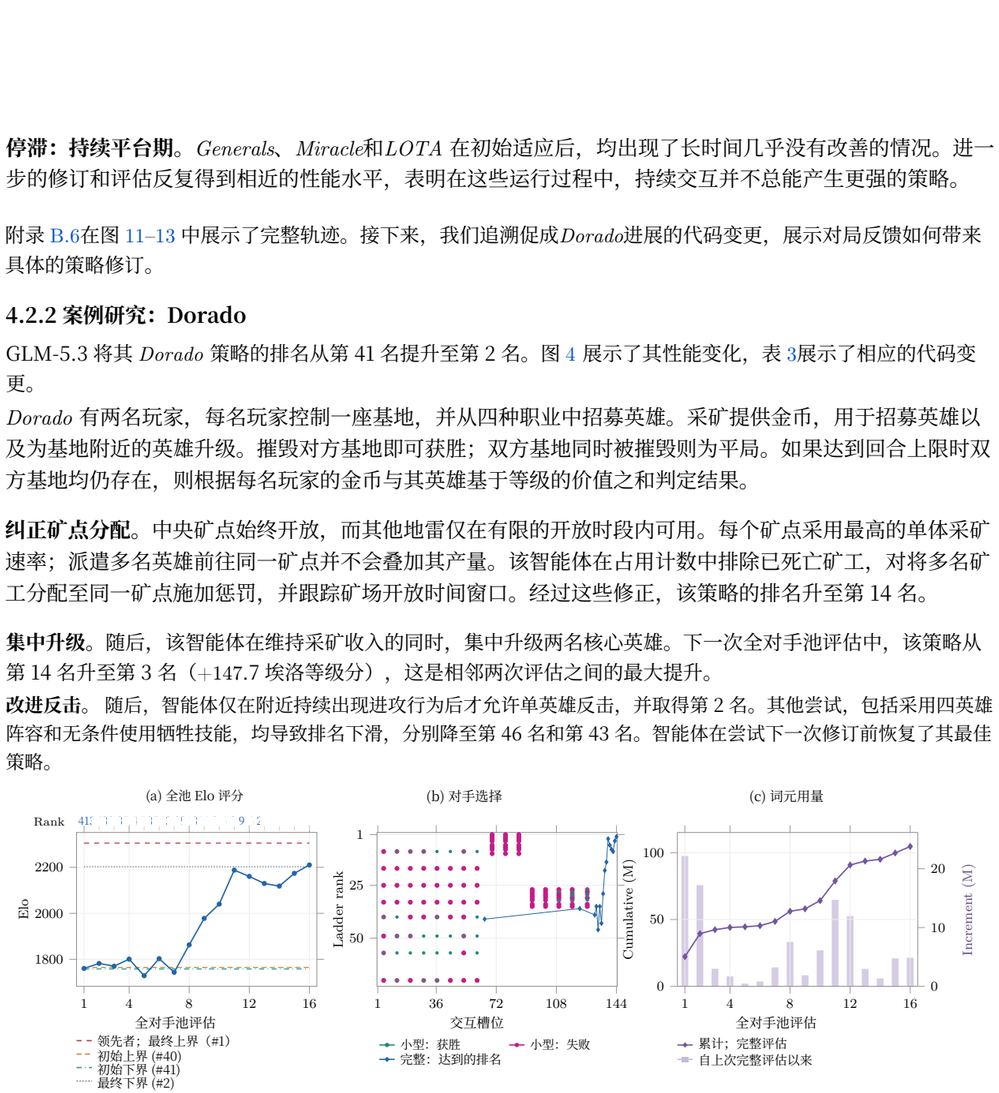

图 4 GLM-5.3 在 *Dorado* 上的表现。128/16 预算下的全池 Elo 评分与排名、小规模—全量交互顺序及词元使用量。附录 B.5 详细说明了预算槽位安排、批次标记、排名标注和词元核算。

Table 3 *Dorado* champion milestones. Mechanism descriptions with added (+) and deleted (-) C++ diff excerpts. Ellipses omit context.

Full-pool eval.	Elo	Rank	Policy adjustment	Code excerpt
2	1782.0	36	Keep **mining**; attack only a vulnerable **enemy base**.	+232 enemyBase != NULL && enemyBase->hp < 1800 && (int)myHeroes.size() >= 4;
4	1800.7	35	Use all **living heroes** to defend a serious threat.	+245 int count = seriousThreat ? (int)defenders.size() : 2;
6	1802.8	35	Trigger **full defence** for active threats or recent damage.	+228 const bool seriousThreat = ... || baseThreat || recentBaseHit;
8	1862.8	29	Exclude **dead miners** from occupancy; spread assignments.	+343 if (alive && mineUsable(it->second, round)) occ[it->second]++; -477 ... + 600LL * occ[m]; +472 ... + 2000LL * occ[m];
9	1977.9	18	Penalize duplicate **mine assignments** more strongly.	-472 ... + 2000LL * occ[m]; +472 ... + 5000LL * occ[m];
10	2040.1	14	Track **mine opening windows** and prioritize **open mines**.	+217 s_mineExistUntil[i] = round + exist->val[0] - 1; + 36 #define MINE_OPEN_OCCUPANCY_PENALTY 50000LL
11	2187.8	3	Concentrate **upgrades** on a **two-hero core**; preserve **mining income**.	+296 if (s_dutyId < 0 && (int)s_leveledCore.size() < 2 ...) +316 if (!coreHammerguard && type == "hammerguard") priority = 0;
16	2210.3	2	Allow **one-hero counterattacks** only after sustained aggression.	+410 s_aggressionRounds = tightAggression ? s_aggressionRounds + 1 : 0; +412 const bool corePush = activeCoreHeroes >= 2 && aggressionThreat;

4.3 Does More Interaction Overcome Performance Plateaus?

We test whether more interaction resolves GLM-5.3's weak results on *Generals*, *Miracle*, *LOTA*, and *AntWar2*. Each continuation keeps the existing workspace and evaluation ledger and adds 256 small-match units and 32 full-pool evaluations. The total budget is 384/48, three times the original 128/16. Table 4 compares the best policies at both budget levels.

Table 4 Threefold-budget continuation for GLM-5.3. Elo and rank of the retained champion at each budget. The extension adds 256 small units and 32 full-pool evaluations to the same run. A positive ΔRank indicates an improved position.

	128 / 16		384 / 48		Change	
Game	Elo	Rank	Elo	Rank	ΔElo	ΔRank
Generals	1441.2	17	1441.2	17	+0.0	+0
Miracle	1436.8	26	1486.8	19	+50.0	+7
LOTA	2147.4	22	2147.4	22	+0.0	+0
AntWar2	1977.1	9	2117.0	6	+139.8	+3

Tripling the budget leaves two retained champions unchanged and improves two; none reaches rank 1 (Table 4). *Generals* and *LOTA* keep their earlier best scores, while *Miracle* moves from rank 26 to 19 and *AntWar2* from rank 9 to 6. Figure 5 shows flat intervals and fluctuations through the extended runs. These outcomes describe a finite continuation of four GLM-5.3 runs under the same development protocol.

Takeaway. Tripling the interaction budget yields limited further improvement, suggesting that the default budget brings these runs close to a performance plateau. Within the tested range, more interaction alone does not overcome the remaining performance gaps.

4.4 Which Feedback Choices Affect Policy Improvement?

We select one representative game from each trajectory category: *Pacman* (Success), *AntWar* (Progress), and *Miracle* (Stagnation). All ablations use GLM-5.3 with max reasoning effort and the same 128-small/16-full budget. Each axis varies one component while keeping its remaining settings fixed. We report retained-champion Elo, with exact Elo/rank tables in Appendix C.

表 3 *Dorado* 冠军里程碑。机制描述，附带标示新增（+）和删除（-）内容的 C++ 差异摘录 s.
省略号表示省略的上下文。

完整池评估	Elo	Rank	策略调整	代码片段
2	1782.0	36	继续采矿；仅攻击易受攻击的敌方基地。	+232 enemyBase != NULL && enemyBase->hp < 1800 && (int)myHeroes.size() >= 4;
4	1800.7	35	使用所有存活英雄抵御严重的威胁。	+245int count =seriousThreat ? (int)defenders.size() : 2;
6	1802.8	35	在存在当前威胁或近期伤害。	+228const bool seriousThreat = ... || baseThreat || recentBaseHit;
8	1862.8	29	在占用统计中排除已死亡矿工；分散任务分配。	+343if (alive && mineUsable(it->second, round)) occ[it->second]++; -477 ... + 600LL * occ[m]; +472 ... + 2000LL * occ[m]; -472 ... + 2000LL * occ[m]; +472 ... + 5000LL * occ[m];
9	1977.9	18	对重复矿场分配施加惩罚力度更大。	
10	2040.1	14	跟踪矿场开放时间窗口，并优先考虑已开放矿场。	+217 s_mineExistUntil[i] = round + exist->val[0]- 1; + 36 #define MINE _OPEN_ 占用 _惩罚 50000LL
11	2187.8	3	将升级集中于双英雄组合 core; prese rve 采矿 g income.	+296if (s_dutyId < 0 && (int)s_leveledCore.size() < 2...) +316if (!coreHammerguard && type ==\"hammerguard\") priority = 0;
16	2210.3	2	仅允许单英雄反击在持续进攻之后。	+410 s_aggressionRounds = tightAggression ? s_aggressionRounds + 1 : 0; +412const bool corePush = activeCoreHeroes >= 2 && aggressionThreat;

4.3 更多交互能否突破性能瓶颈?

我们测试增加交互是否能改善 GLM-5.3 在*Generals*、*Miracle*、*LOTA*和*AntWar2*中的不佳表现。每次续跑均保留现有工作区和评估台账，并增加 256 个小规模对局单元和 32 次全池评估。总预算为 384/48，是原始预算 128/16 的三倍。表 4 比较了两种预算水平下的最佳策略。

表 4 GLM-5.3 的三倍预算续跑。各预算下保留的冠军的埃洛等级分和排名。扩展阶段在同一次运行中额外增加了 256 个小型单元和 32 次全池评估。 Δ排名为正值表示名次提升。

	128 / 16		384 / 48		变化	
Game	Elo	Rank	Elo	Rank	ΔElo	Δ排名
Generals	1441.2	17	1441.2	17	+0.0	+0
Miracle	1436.8	26	1486.8	19	+50.0	+7
LOTA	2147.4	22	2147.4	22	+0.0	+0
AntWar2	1977.1	9	2117.0	6	+139.8	+3

将预算增至三倍后，两个保留的冠军保持不变，另两个有所提升；但均未达到排名第1（表4）。*Generals* 和 *LOTA* 保持了此前的最佳得分，而*Miracle* 从第26名升至第19名，*AntWar2*从第9名升至第6名。图 5 展示了延长运行期间的平台区间与波动。这些结果反映了在相同开发协议下，对四次 GLM-5.3 运行进行有限延续的情况。

要点。将交互预算增至三倍仅带来了有限的进一步改进，表明默认预算已使这些运行接近性能平台期。在测试范围内，仅增加交互并不能弥合剩余的性能差距。

4.4 哪些反馈选择会影响策略改进?

我们从每个轨迹类别中选择一个代表性游戏：吃豆人（成功）、*AntWar*（进展）和*Miracle*（停滞）。所有消融实验均使用 GLM-5.3，将推理强度设为最高，并采用相同的128次小规模运行/16次完整运行预算。每个实验维度仅改变一个组件，其余设置保持不变。我们报告保留冠军的Elo评分，详细的埃洛等级分/排名表见附录 C。

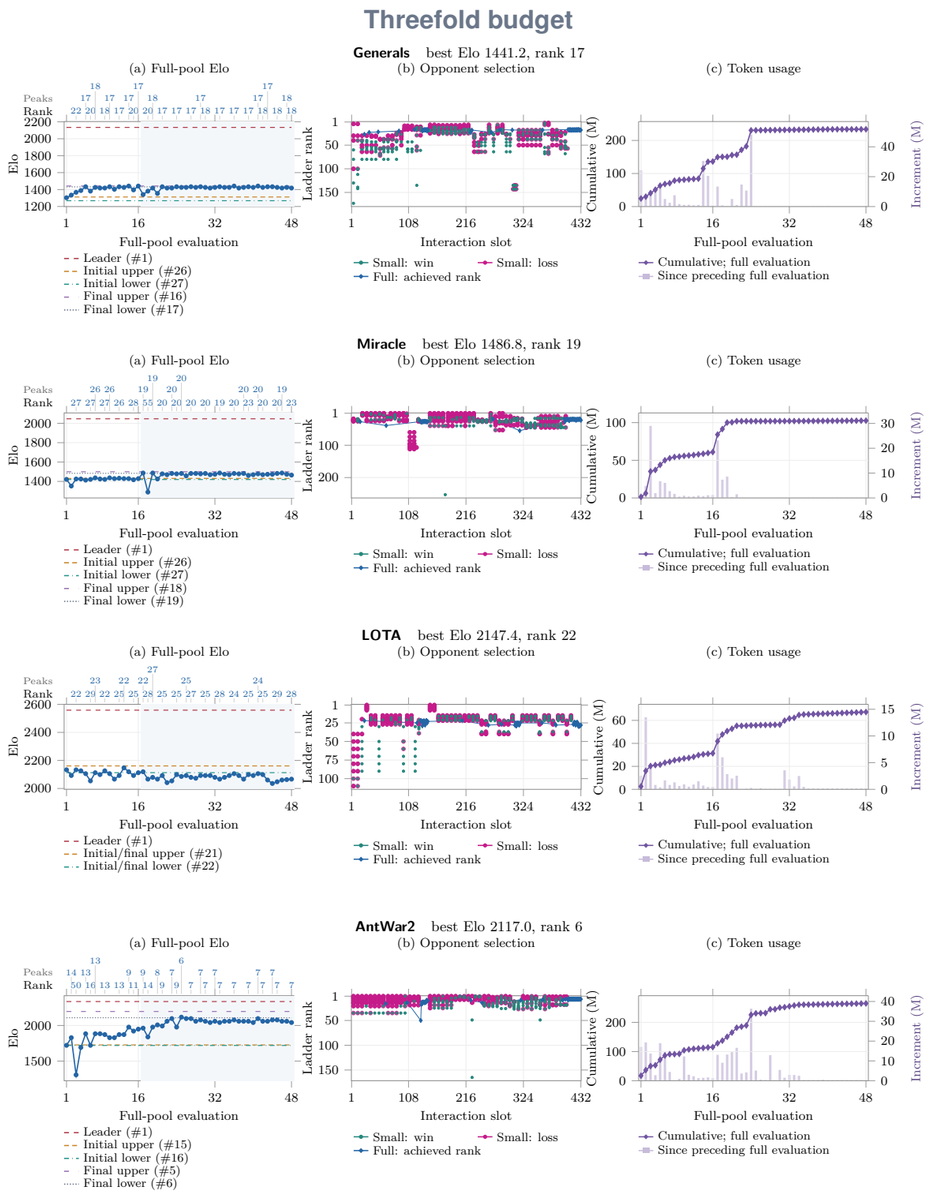

Figure 5 Policy performance under a threefold budget. Full-pool Elo, interaction order, and token use for four continued runs. Ranks appear at every third full-pool evaluation and local Elo peaks. Plot conventions appear in Appendix B.5.

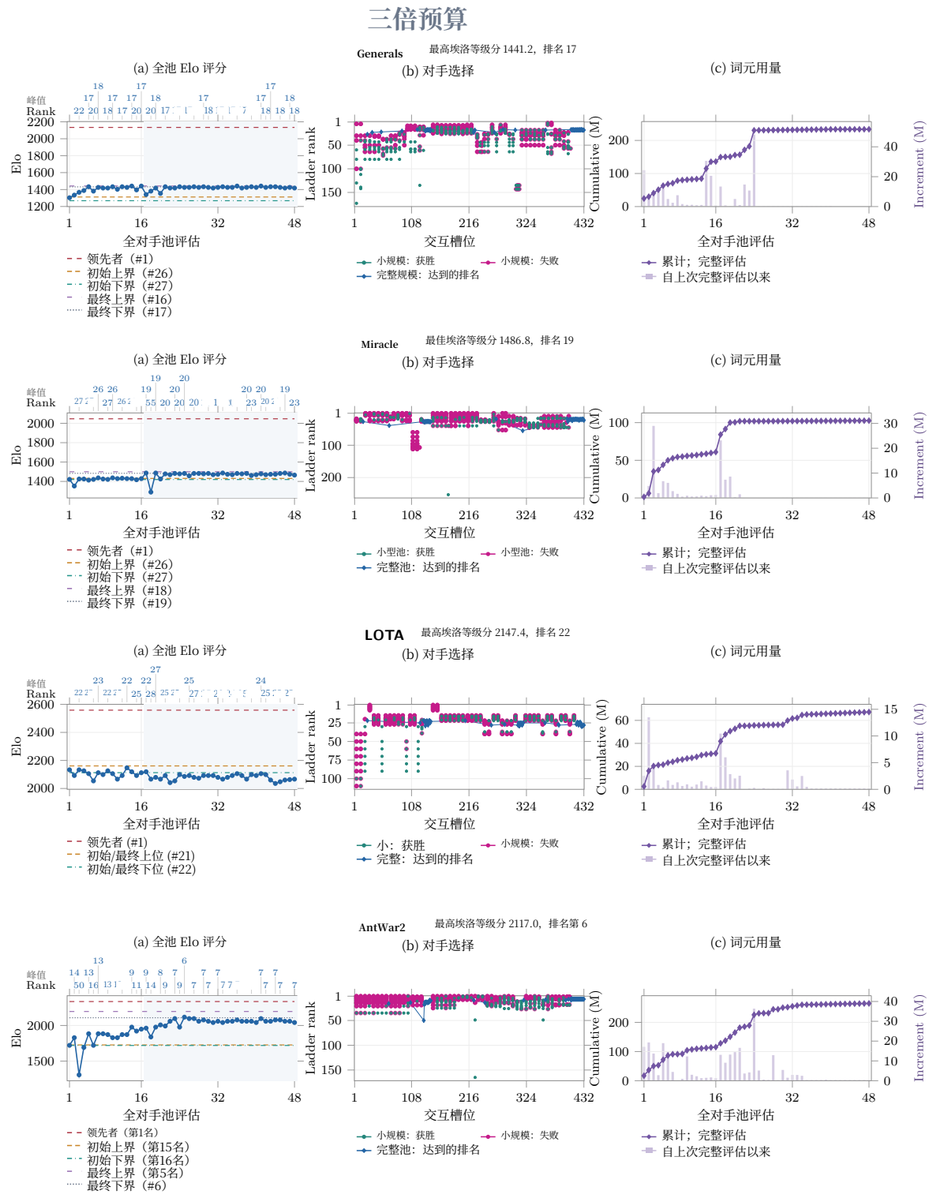

图 5 三倍预算下的策略性能。四次延续运行的全池 Elo 评分、交互顺序和词元使用量。每隔三次全对手池评估以及在局部Elo峰值处标注排名。绘图规范见附录 B.5。

4.4.1 Does Opponent Selection Affect Performance?

We compare four opponent-selection strategies. *Model selection* lets the agent choose opponents, as in the main experiment. The *ladder* starts at rank 30 and moves toward stronger opponents without returning to weaker ones. *Random sampling* draws uniformly from the pool; *top-four* uses only the four highest-ranked opponents. All conditions use dense feedback and a small-match batch size of four.

The ladder leads on *Pacman* and *AntWar*, while model selection leads on *Miracle* (Figure 6). Model selection outperforms random and top-four selection in all three games; the ladder trails random selection on *Miracle*. Top-four selection yields the lowest Elo throughout. One hypothesis is that gradually stronger or agent-selected opponents provide more actionable feedback than repeated mismatches.

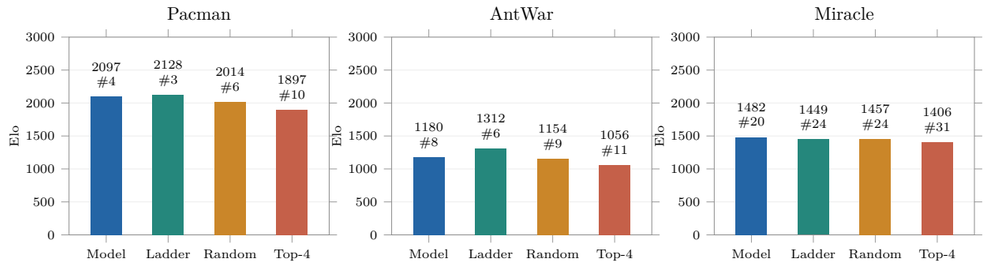

Figure 6 Opponent selection. GLM-5.3 retained-champion Elo and rank with model, ladder, random, and top-four selection; all use dense feedback and batch size 4. Plot conventions appear in Appendix B.5.

Takeaway. Opponent selection is important for effective policy development: agents need opponents whose matches provide useful information for improvement. Agent-directed selection and progressively stronger opponents both perform well, showing the value of choosing opponents for learning.

4.4.2 Does Dense Replay Feedback Affect Performance?

We compare detailed small-match replays and their translations with binary win/non-win feedback. Both conditions keep conversation history, agent-selected opponents, and full-pool feedback; the agent chooses the batch size.

Dense feedback yields higher retained-champion Elo in all three games, with gains that vary by game (Figure 7). Detailed replays supply state, action, and consequence information for diagnosing decision errors; binary feedback supplies the outcome alone. The gains support retaining this diagnostic context in AHL under the tested budgets.

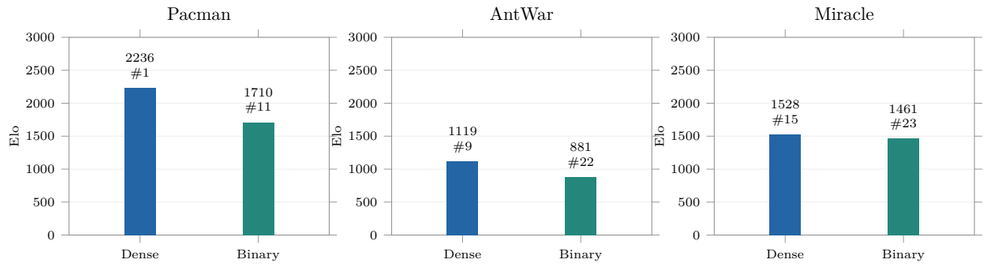

Figure 7 Replay feedback. Dense replays versus binary win/non-win feedback under the same 128/16 budget. Labels give Elo and rank. Dense feedback improves performance in all three representative games.

4.4.1 对手选择是否影响性能?

我们比较了四种对手选择策略。模型选择与主实验一样，让智能体自行选择对手。阶梯式选择从排名第30的对手开始，逐步转向更强的对手，不再回头选择较弱的对手。随机采样从对手池中均匀抽取对手；前四名选择仅使用排名最高的四个对手。所有条件均使用密集反馈，小型对局批大小均为四。

阶梯式选择在吃豆人和*AntWar*中领先，而模型选择在*Miracle* 中领先（图6）。在全部三款游戏中，模型选择均优于随机选择和前四名选择；在*Miracle*中，阶梯式选择落后于随机选择。前四名选择的埃洛等级分始终最低。一种假设是，与反复进行实力悬殊的对局相比，逐渐增强的对手或由智能体自行选择的对手能够提供更具可操作性的反馈。

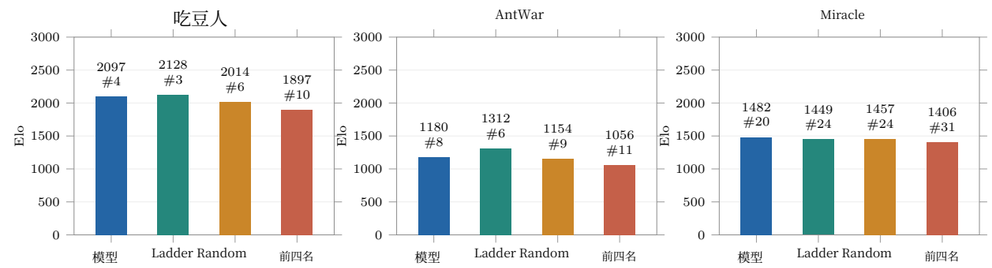

图6 对手选择。GLM-5.3 在模型选择、阶梯式选择、随机选择和前四名选择下保留冠军的Elo评分与排名；所有条件均采用密集反馈，批大小为4。绘图约定见附录 B.5。

要点。对手选择对于有效的策略开发至关重要：智能体需要能够通过对局提供有用改进信息的对手。智能体主导的选择和逐步增强的对手均表现良好，说明了为学习而选择对手的价值。

4.4.2 稠密回放反馈是否影响性能?

我们将详细的小型对局回放及其转译内容与胜利/非胜利二元反馈进行比较。两种条件均保留对话历史、由智能体选择的对手以及全对手池反馈；批大小由智能体选择。

在全部三款游戏中，密集反馈均带来了更高的保留冠军的Elo评分，提升幅度因游戏而异（图 7）。详细的对局回放提供了状态、动作及其后果的信息，可用于诊断决策错误；二元反馈则仅提供结果。这些提升表明，在所测试的预算下，应在 AHL 中保留此类诊断上下文。

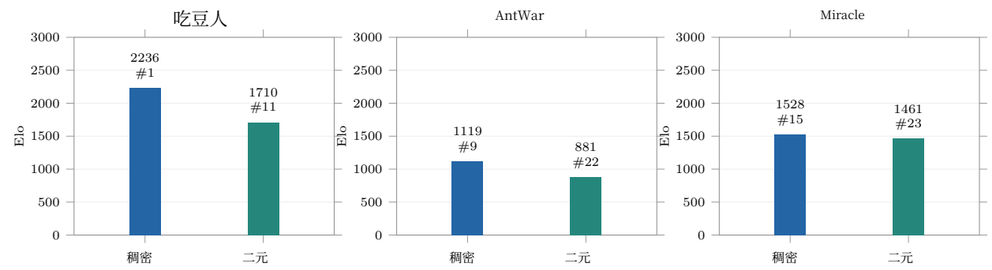

图7 回放反馈。在相同的128/16预算下，比较密集回放与胜利/非胜利二元反馈。标签标示埃洛等级分和排名。密集反馈在三个代表性游戏中均提升了性能。

Takeaway. A wider feedback channel enables more efficient policy development. Dense replays provide decision-level information that agents can use to diagnose failures and guide code revisions, improving performance under a fixed budget.

4.4.3 Does the Number of Opponents per Small Match Matter?

We compare 1, 2, 4, and 8 opponents per small-match submission, keeping model selection and dense feedback. The total allowance remains 128 opponent tests and 16 full-pool evaluations.

Intermediate batches perform best: four opponents on *Pacman* and *Miracle*, and two on *AntWar* (Figure 8). Eight opponents per request spend the small-match allowance in 16 full-sized requests, compared with 32–64 requests for batches of 2–4. The ranking is consistent with a trade-off between matchup breadth per request and opportunities to test revised policies. The observed preference for 2–4 opponents concerns these three games and settings.

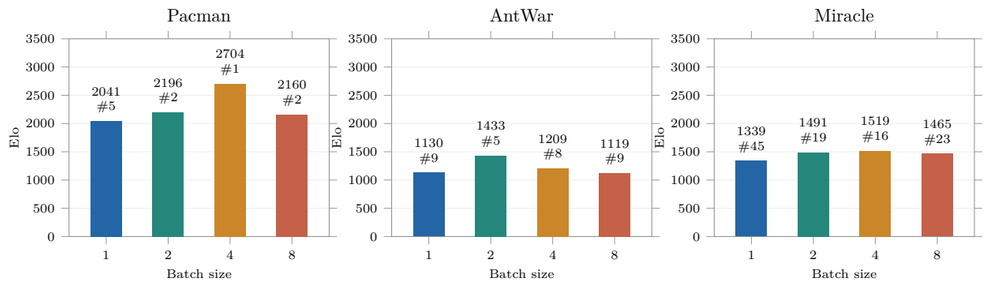

Figure 8 Small-match batch size. Number of opponents per submission, with a fixed total allowance of 128 opponent tests and 16 full-pool evaluations. Labels give Elo and rank. The best size is 4, 2, and 4 for *Pacman*, *AntWar*, and *Miracle*, respectively.

Takeaway. The ablations show that opponent selection, replay detail, and batch size all affect retained-champion performance under a fixed budget. We next vary which players generate the replay evidence.

4.5 Can Agents Learn from Off-Policy Replays?

We test whether *off-policy* replays from matches between other policies support policy improvement. On *Pacman*, *AntWar*, and *Miracle*, GLM-5.3 chooses replays from a catalog of matches between frozen-pool opponents. It can request 128 replay views and 16 full-pool evaluations of its own policy. Reasoning effort, game resources, opponent ladders, and budget ceilings match the on-policy setting. Only the replay source changes; the agent still receives full-pool feedback on its own policy.

All three off-policy runs improve from their first evaluated policy to the retained champion: *Pacman* advances from rank 10 to 1, *AntWar* from 41 to 19, and *Miracle* from 27 to 15 (Figure 9).

Miracle provides the clearest contrast with learning from the agent's own matches. Its on-policy trajectory plateaus with a retained champion at rank 26, while the independent off-policy run reaches rank 15 (Figure 10). This contrast suggests that other players' experience can support progress beyond the plateau observed with self-generated replays. On *Pacman*, both settings reach rank 1; on *AntWar*, on-policy learning performs better, reaching rank 5 compared with 19 off-policy.

要点。更宽的反馈通道能够提高策略开发效率。密集回放提供了决策级信息，智能体可利用这些信息诊断失败原因并指导代码修订，从而在固定预算下提升性能。

4.4.3 每次小规模对局的对手数量是否重要?

我们比较了每次小规模对局提交包含 1、2、4 和 8 个对手的情况，同时保留模型选择和密集反馈。总配额保持为 128 次对手测试和 16 次全池评估。

中等批大小的表现最佳：在 吃豆人 和 *Miracle* 中为四个对手，在 *AntWar* 中为两个对手（图 8）。每次请求包含八个对手时，16 次满额请求便会耗尽小规模对局配额；相比之下，批大小为 2–4 时可进行 32–64 次请求。这一排名与每次请求的对局覆盖广度和测试修订后的策略的机会之间的权衡相吻合。观察到的对 2–4 个对手的偏好仅针对这三个游戏及其设置。

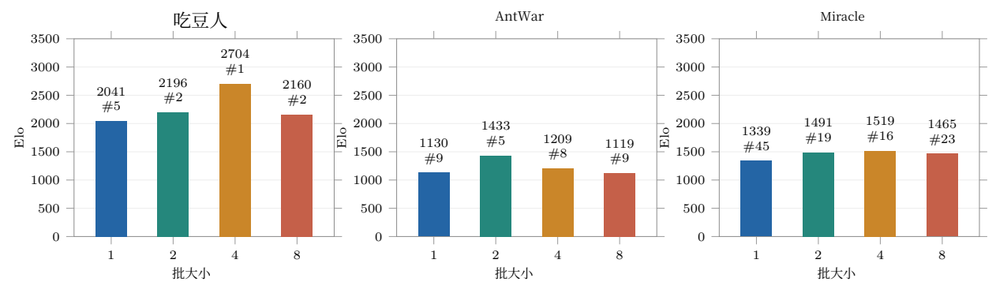

图 8 小型对局批大小。每次提交的对手数量，总配额固定为 128 次对手测试和 16 次全池评估。标签给出了埃洛等级分和排名。吃豆人、*AntWar* 和 *Miracle* 的最佳批大小分别为 4、2 和 4。

要点。消融实验表明，在固定预算下，对手选择、回放细节和批大小都会影响保留冠军的表现。接下来，我们将改变生成回放证据的玩家。

4.5 智能体能否从异策略对局回放中学习?

我们测试其他策略之间的对局所产生的异策略对局回放是否有助于策略改进。在吃豆人、*AntWar*和 *Miracle*中，GLM-5.3 从冻结对手池中的对手之间的对局目录中选择对局回放。它可以请求 128 次回放查看，以及对其自身策略进行 16 次全池评估。推理强度、游戏资源、对手天梯和预算上限均与同策略设置保持一致。唯一改变的是回放来源；智能体仍会收到针对其自身策略的全对手池反馈。

三次离策略运行从首次已评估策略到保留冠军均有所提升：吃豆人从第 10 名升至第 1 名，*AntWar* 从第 41 名升至第 19 名，*Miracle* 从第 27 名升至第 15 名（图 9）。

Miracle 最清晰地体现了与从智能体自身对局中学习的对比。其同策略轨迹进入平台期，保留的冠军排名为第26，而独立的异策略运行则达到第15名（图10）。这一对比表明，其他玩家的经验能够推动进展，突破使用自生成回放时观察到的平台期。在吃豆人中，两种设置均达到第1名；在 *AntWar* 中，同策略学习表现更好，达到第5名，而异策略学习则为第19名。

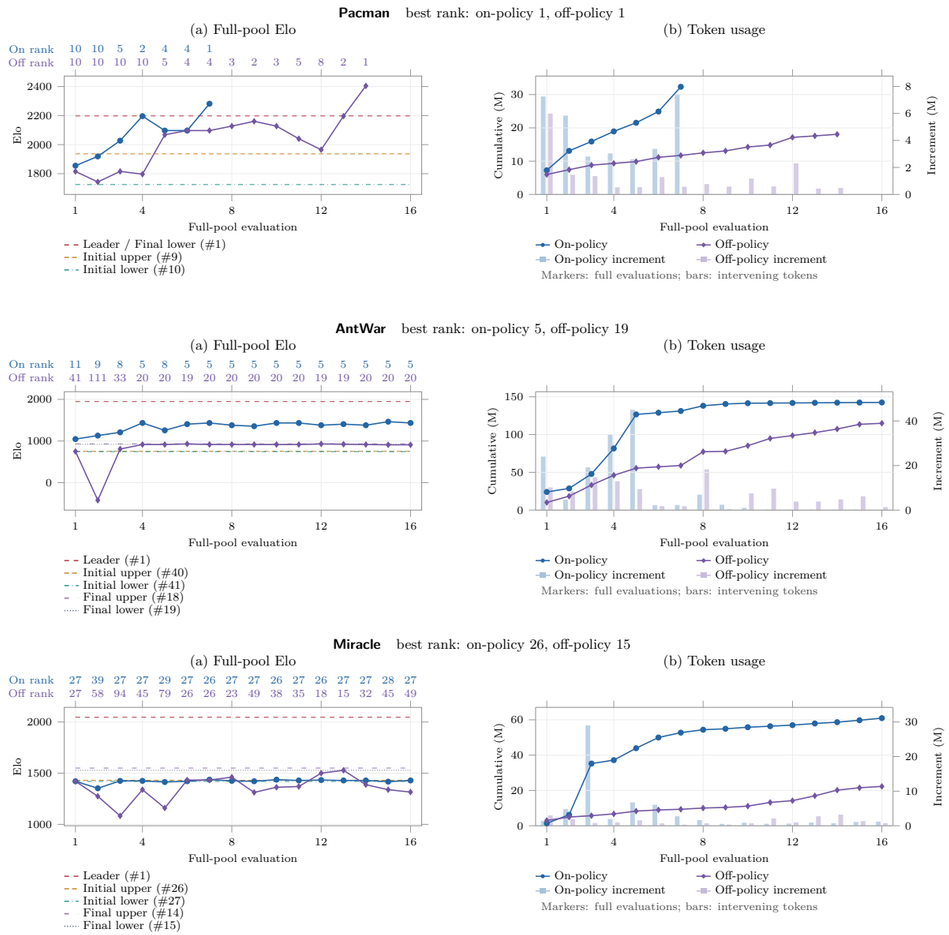

Figure 9 On-policy and off-policy learning trajectories. Full-pool Elo and ranks, cumulative tokens, and token increments for GLM-5.3 with self-generated and other players' replays. Both conditions receive full-pool feedback on their own policies. Appendix B.5 specifies rank rows, reference levels, and token accounting.

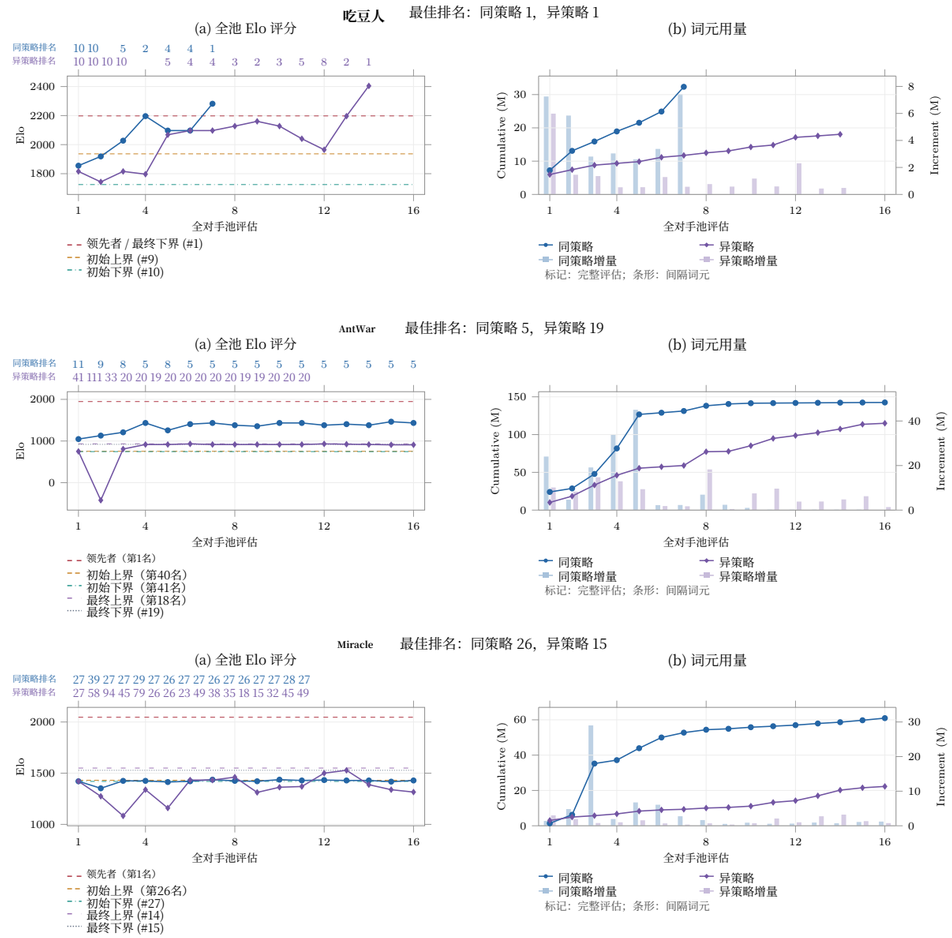

图 9 同策略学习与异策略学习轨迹。GLM-5.3 使用自身生成的对局回放和其他玩家的对局回放时的全池 Elo 评分与排名、累计词元数及词元增量。两种条件下均会收到针对自身策略的全对手池反馈。附录 B.5 详细说明了排名行、参考水平和词元核算。

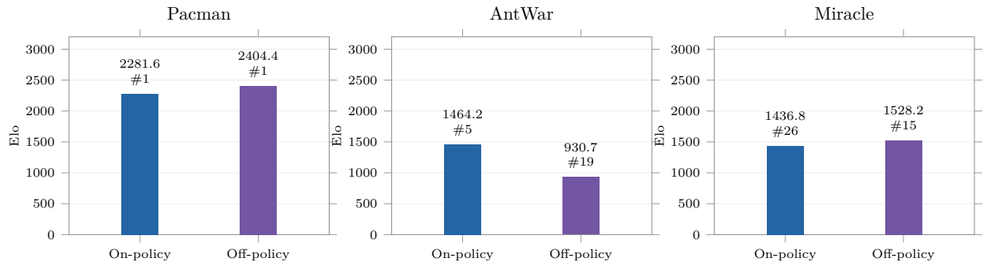

Figure 10 On-policy versus off-policy replay learning. Retained-champion Elo and rank for GLM-5.3 under the same 128/16 budget ceilings. On-policy scores are the main-table results. Labels give Elo and rank; each panel uses the same zero-based Elo scale. Both *Pacman* runs terminate early upon reaching rank 1.

Takeaway. Off-policy replays broaden the experience available to AHL and can support progress beyond the plateaus observed with on-policy learning. Other players' experience can suggest alternative strategies, with the potential to outperform learning from the agent's own matches.

5 Related Work

Table 5 Representative game-agent evaluation protocols. Feedback denotes information available to the evaluated AI agent: ✓ indicates availability and a dash indicates absence in the named setting. Traces include in-game observations or post-match logs. CodeClash refers to its standard, closed-code tournament; GACL to code generation.

Work	AI output	Opponents	Feedback		Protocol
			Scores	Traces	
Direct play					
GameBench (Costarelli et al., 2024)	Actions	Human + AI players	✓	✓	Direct play
TextArena (competitive) (Guertler et al., 2025)	Actions	Human + AI players	✓	✓	Direct play
Program development					
GACL (Game Agent Coding League, n.d.)	Programs	AI-developed programs	–	–	One-shot
Vibe-coding (IR) (Danassis and Goel, 2025)	Programs	Human-written + AI-developed programs	✓	–	Preset
CATArena (games) (Fu et al., 2026)	Programs	AI-developed programs	✓	✓	Preset
CodeClash (tournament) (Yang et al., 2026a)	Programs	AI-developed programs	✓	✓	Preset
CC:Ladder (Yang et al., 2026a)	Programs	Human-written programs	✓	✓	Preset
EvoPolicyGym (Wang et al., 2026)	Programs	None	✓	✓	Agent-scheduled
AAArena (ours)	Programs	Human-written programs	✓	✓	Agent-scheduled

Programs are executable code policies. Preset means fixed refinement rounds, tournament rounds, or opponent order; Agent-scheduled means that the agent chooses evaluation requests within a budget. These labels describe the outer schedule. Scores include outcomes, returns, and rankings; traces include observations, match logs, and replays. CC:Ladder starts from a human solution. IR denotes iterative refinement. Rows describe specific evaluation conditions: GACL covers code generation, and CodeClash covers its default code-opaque tournaments. EvoPolicyGym budgets single-agent episodes and hides validation feedback; AAArena uses frozen human programs and returns full-pool scores during development under a separate evaluation budget.

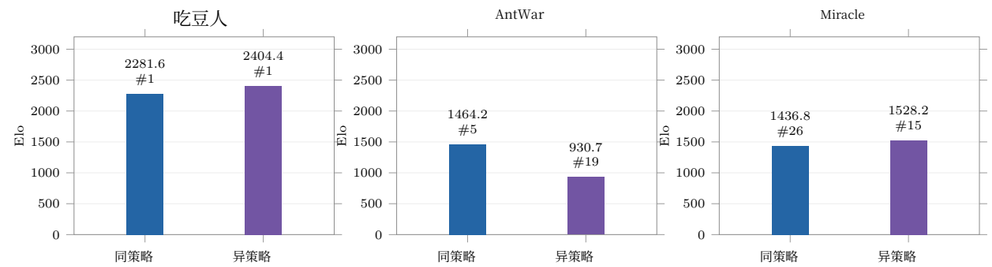

图10 同策略与离策略回放学习对比。在相同的128/16预算上限下，GLM-5.3保留冠军的Elo评分及排名。同策略得分为主表中的结果。标签标示埃洛等级分和排名；各面板均采用相同的、以零为起点的Elo评分尺度。两次吃豆人运行均在达到第1名后提前终止。

要点。异策略回放拓宽了 AHL 可利用的经验范围，并有助于突破同策略学习中观察到的平台期。其他玩家的经验可以启发不同的策略，其效果有可能优于从智能体自身的对局中学习。

5 相关工作

表5 代表性的游戏智能体评估协议。反馈指被评估的人工智能智能体可获取的信息： ✓ 表示在所述设置中可获取，短横线表示不可获取。轨迹包括游戏内观测或赛后日志。CodeClash指其标准的闭源代码锦标赛；GACL指代码生成。

Work	人工智能输出	对手	反馈		协议
			得分	轨迹	
直接对弈					
GameBench（Costarelli 等人，2024年）	动作	人类 +AI 玩家	✓	✓	直接对弈
TextArena（竞技）（Guertler 等人，2025年）	动作	人类 + AI 玩家	✓	✓	直接对弈
程序开发					
GACL（GameAgent Coding League，无日期）	程序	人工智能开发的程序	–	–	单次生成
氛围编程（迭代改进）（Danassis 和 Goel，2025年）	程序	人工编写 + 人工智能开发的程序	✓	–	预设
CATArena（游戏）（Fu 等人，2026年）	程序	人工智能开发的程序	✓	✓	预设
CodeClash（锦标赛）（Yang 等人，2026a）	程序	人工智能开发的程序	✓	✓	预设
CC:Ladder（Yang 等人,2026a）	程序	人工编写程序	✓	✓	预设
EvoPolicyGym（Wang 等人，2026年）	程序	None	✓	✓	智能体调度
AAArena（本文）	程序	人工编写程序	✓	✓	智能体调度

程序是可执行代码策略。“预设”表示固定的改进轮次、锦标赛轮次或对手顺序；“智能体调度”表示智能体在预算内选择评估请求。这些标签描述的是外层调度。得分包括结果、回报和排名；轨迹包括观测、对局日志和对局回放。CC:Ladder 从人工解决方案开始。迭代改进（IR）指迭代改进。各行描述具体的评估条件：GACL 涵盖代码生成，CodeClash 涵盖其默认的代码不透明锦标赛。EvoPolicyGym 为单智能体回合设置预算，并隐藏验证反馈；AAArena 使用冻结的人工程序，并在开发期间根据单独的评估预算返回全池得分。

Competitive program development. Competitive program-development benchmarks differ in their outputs, opponents, feedback, and revision protocols (Table 5). GACL evaluates generated game agents and ProxyWar includes code repair (Game Agent Coding League, n.d.; Peng et al., 2026); the vibe-coding tournament compares student and model programs with a self-play refinement condition (Danassis and Goel, 2025). CATArena supplies tournament logs and peer code across revision rounds (Fu et al., 2026), while CodeClash studies competitive code evolution and CC:Ladder advances through ranked human programs after each win (Yang et al., 2026a). EvoPolicyGym budgets episodes in single-agent environments and uses hidden validation (Wang et al., 2026). AAArena combines 12 custom competition games, archived human-written programs, and agent-selected testing, with separate budgets for small matches and full-pool evaluations.

HL and program evolution. Weng's HL incorporates experience through revisions to program policies and supporting software (Weng, 2026). Programmatic reinforcement learning and LLM-guided program search also use programs as policy representations (Verma et al., 2018; Bastani et al., 2018; Eberhardinger et al., 2025; Kuang et al., 2025). Related systems improve controllers, algorithms, and agent software (Garnier et al., 2026; Romera-Paredes et al., 2023; Novikov et al., 2025; Hu et al., 2025c; Zelikman et al., 2024), or revise rewards and training procedures (Ma et al., 2024; Chi et al., 2026). Trajectory distillation extracts reusable skills from replay (Yang et al., 2026b). AHL applies HL to competitive game-agent development, where the AI agent revises policies and supporting software with a fixed base model.

Direct game-playing evaluation. Direct game-playing benchmarks measure the model's decisions during matches. BALROG and LMGame-Bench evaluate action selection in interactive games (Paglieri et al., 2025; Hu et al., 2025a). GameBench, GameArena, TextArena, GAMEBoT, ZeroSumEval, and Kaggle Game Arena extend this evaluation to competitive players (Costarelli et al., 2024; Hu et al., 2025b; Guertler et al., 2025; Lin et al., 2025; Khan et al., 2025; Doerschuk-Tiberi et al., 2026). These settings measure the player, whereas AAArena measures the AI agent developing the autonomous program that plays the game.

6 Future Work

Extending AAArena. We plan to add games, controlled rule variants, and changing opponent populations to study adaptation to changes in the task and competition. These extensions would broaden evaluation beyond per-game rankings to adaptation speed, cross-game transfer, and continual learning.

Heuristic and reinforcement learning. We aim to compare how code revisions and parameter updates learn from experience and assign credit for improvements. Such comparisons should account for feedback access and interaction budgets. Hybrid learners could use HL to revise policy structure and reinforcement learning to train its components.

From replay use to active learning. External replays could help agents form strategic hypotheses before choosing opponents to test them. We plan to study how agents allocate replay views and matches to resolve uncertainty. Agents could turn validated strategies into reusable policy modules.

7 Conclusion

We present AAArena to evaluate AI agents that develop executable game policies through adversarial heuristic learning (AHL). The benchmark combines 12 real-world competition games, archived human programs, and a unified two-budget protocol. Across 7 AI models, Opus5.5 with Claude Code earns 6 gold medals; no evaluated configuration tops the remaining 6 human ladders. Dense feedback supports policy improvement in the ablations, and agents also improve by learning from both on-policy and off-policy replays. Qualitative analysis of code revisions and trajectories points to challenges in game understanding, strategy implementation, and long-horizon planning. By making this development process measurable, AAArena provides a foundation for research on heuristic learning and self-evolving AI agents.

竞争性程序开发。竞争性程序开发基准测试在输出、对手、反馈和修订协议方面各有不同（表5）。GACL 评估生成的游戏智能体，ProxyWar 则包含代码修复（Game Agent CodingLeague，无日期；Peng 等人，2026年）；氛围编程锦标赛比较学生和模型编写的程序，并设置了自博弈优化条件（Danassis 和 Goel，2025年）。CATArena 在各修订轮次提供锦标赛日志和同伴代码（Fu 等人，2026年），而 CodeClash 研究竞争性代码演化，CC:Ladder 则在每次获胜后，依次挑战排名更高的人类编写的程序（Yang 等人，2026a）。EvoPolicyGym 为单智能体环境中的回合分配预算，并采用隐藏验证（Wang 等人，2026年）。AAArena 将 12 款定制竞赛游戏、存档的人类编写的程序和智能体选择的测试相结合，并为小规模对局和全池评估分别设置预算。

启发式学习与程序演化。Weng 的启发式学习通过修改程序策略和配套软件来融入经验（Weng, 2026年）。程序化强化学习和大语言模型引导的程序搜索也使用程序作为策略表示（Verma 等人, 2018年; Bastani 等人, 2018年; Eberhardinger 等人 , 2025 年 ; Kuang 等人 , 2025 年）。相关系统改进控制器、算法和智能体软件（Garnier 等人, 2026年; Romera-Paredes 等人, 2023年; Novikov 等人, 2025年; Hu 等人, 2025年c; Zelikman 等人, 2024年），或修改奖励和训练流程（Ma 等人, 2024年; Chi 等人, 2026年）。轨迹蒸馏从回放中提取可复用技能（Yang 等人, 2026年b）。AHL 将启发式学习应用于竞争性游戏智能体开发，在此过程中，人工智能智能体在基础模型固定不变的情况下修改策略和配套软件。

直接游戏对战评估。直接游戏对战基准测试衡量模型在对局中的决策。BALROG 和 LMGame-Bench 评估交互式游戏中的动作选择（Paglieri 等人，2025年；Hu 等人，2025年a）。GameBench、GameArena、TextArena、GAMEBoT、ZeroSumEval 和 Kaggle Game Arena 将这一评估扩展至竞技玩家（Costarelli 等人，2024年；Hu 等人，2025年b；Guertler 等人，2025年；Lin 等人，2025年；Khan 等人，2025年；Doerschuk-Tiberi 等人，2026年）。这些设置评估的是玩家，而 AAArena 评估的是开发用于进行游戏的自主程序的人工智能智能体。

6 未来工作

扩展 *AAArena*。我们计划增加游戏、受控规则变体以及动态变化的对手群体，以研究对任务与竞争环境变化的适应能力。这些扩展将使评估不再局限于单游戏排名，进一步涵盖适应速度、跨游戏迁移和持续学习。

启发式学习与强化学习。我们旨在比较代码修订和参数更新如何从经验中学习，以及如何为改进分配功劳。此类比较应考虑反馈获取和交互预算。混合学习器可以使用启发式学习来修订策略结构，并使用强化学习来训练其组件。

从回放使用到主动学习。外部对局回放可以帮助智能体形成策略假设，然后再选择对手来检验这些假设。我们计划研究智能体如何分配回放查看和对局的预算，以消除不确定性。智能体可以将经过验证的策略转化为可复用的策略模块。

7 结论

我们提出 AAArena，用于评估通过对抗式启发学习（AHL）开发可执行游戏策略的人工智能智能体。该基准测试整合了 12 款真实世界的竞赛游戏、归档人类程序以及统一的双预算协议。在对 7 个人工智能模型的评估中，搭配 Claude Code 的 Opus5.5 获得了 6 枚金牌；在其余 6 个人类玩家天梯中，没有任何受评配置登顶。消融实验表明，密集反馈有助于策略改进，智能体也能通过学习同策略和异策略回放取得进步。对代码修订和轨迹的定性分析揭示了游戏理解、策略实现和长时域规划方面的挑战。通过使这一开发过程可衡量，AAArena 为启发式学习和自我演化人工智能智能体的研究奠定了基础。

Acknowledgement

We thank the many members of the Student Association of the Department of Computer Science and Technology at Tsinghua University for developing and maintaining the Tsinghua University Agent Competition over the years and for supporting this work. We thank Z.ai for sponsoring this work. We also thank all participants in past editions of the Tsinghua University Agent Competition.

References

Osbert Bastani, Yewen Pu, and Armando Solar-Lezama. Verifiable reinforcement learning via policy extraction. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/e6d8545daa42d5ced125a4bf747b3688-Paper.pdf.

Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs. *Biometrika*, 39(3-4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324. URL https://doi.org/10.1093/biomet/39.3-4.324.

Murray Campbell, A.Joseph Hoane, and Feng-hsiung Hsu. Deep blue. *Artificial Intelligence*, 134(1-2):57–83, January 2002. ISSN 0004-3702. doi: 10.1016/s0004-3702(01)00129-1. URL http://dx.doi.org/10.1016/S0004-3702(01)00129-1.

Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. Ai4ai-bench: Benchmarking llm agents in algorithmic design for recursive self-improvement, 2026. URL https://arxiv.org/abs/2608.20318.

Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng, Wenjie Li, Joshua Clymer, and Arjun Yadav. Gamebench: Evaluating strategic reasoning abilities of llm agents, 2024. URL https://arxiv.org/abs/2406.06613.

Rémi Coulom. *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*, pages 72–83. Springer Berlin Heidelberg, 2007. ISBN 9783540755388. doi: 10.1007/978-3-540-75538-8_7. URL http://dx.doi.org/10.1007/978-3-540-75538-8_7.

Panayiotis Danassis and Naman Goel. Can vibe coding beat graduate cs students? an llm vs. human coding tournament on market-driven strategic planning, 2025. URL https://arxiv.org/abs/2511.20613.

Bovard Doerschuk-Tiberi, Yao Yan, Justin Chiu, Hann Wang, Timothy Chung, Martyna Plomecka, John Schultz, Jon Lipovetz, Clayton Drazner, Yuchen Zhuang, Jaimie Hwang, Nate Keating, Riley Jones, Andrew Lee, Oran Kelly, Ian Gemp, Michael Aaron, Laurel Prince, Kate Larson, Jeff Moser, Harrison Jobe, Chad Woodford, Siqi Liu, Andrew Wang, Bo Chang, Christopher D'Mello, Diane Chaleff, Addison Howard, Johnny Yip, Chuck Sugnet, Antonio Gulli, Meghan O'Connell, Will Cukierski, Nenad Tomasev, Dima Yeroshenko, Kinjal Parekh, Roxanne Daniel, Marc Lanctot, Domino Weir, Elsa Dong, Daniel Hennes, Melissa Nalubwama, Robert Fraser, Ryan Trostle, Jun Peng, Tom Mason, Lloyd Hightower, Chiamaka Chukwuka, Yuexiang Zhai, Phoebe Kirk, Yi Su, Yuting Han, Jie Ren, Chris Prichard, Sahand Sharifzadeh, Karim Hakimzadeh, DJ Sterling, Meg Risdal, Kate Olszewska, Ya Xu, Orhan Firat, and Minmin Chen. Game arena: Strategic llm evaluation in competitive environments, 2026. URL https://arxiv.org/abs/2609.31473v1.

Manuel Eberhardinger, James Goodman, Alexander Dockhorn, Diego Perez-Liebana, Raluca D. Gaina, Duygu Çakmak, Setareh Maghsudi, and Simon Lucas. From code to play: Benchmarking program search for games using large language models, 2025. URL https://arxiv.org/abs/2412.04057.

Lingyue Fu, Xin Ding, Linyue Pan, Yaoming Zhu, Shao Zhang, Lin Qiu, Weiwen Liu, Weinan Zhang, Xuezhi Cao, Xunliang Cai, Jiaxin Ding, and Yong Yu. Catarena: Evaluating evolutionary capabilities of code agents via iterative tournaments, 2026. URL https://arxiv.org/abs/2510.26852.

Game Agent Coding League. Game Agent Coding League for LLMs. Official benchmark documentation, n.d. URL https://gameagentcodingleague.com/. Accessed September 28, 2026; publication date not stated.

Paul Garnier, Jonathan Viquerat, and Elie Hachem. Heuristic learning for active flow control using coding agents, 2026. URL https://arxiv.org/abs/2607.11565.

Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. Textarena, 2025. URL https://arxiv.org/abs/2504.11442.

致谢

我们感谢清华大学计算机科学与技术系学生会的众多成员多年来开发和维护清华大学智能体竞赛，并为本研究提供支持。我们感谢 Z.ai 对本研究的赞助。我们也感谢历届清华大学智能体竞赛的所有参赛者。

参考文献

Osbert Bastani、Yewen Pu 和 Armando Solar-Lezama . 通过策略提取实现可验证强化学习 . 载于 S. Bengio、H. Wallach、H. Larochelle、K. Grauman、N. Cesa-Bianchi 和 R. Garnett 主编的 神经信息处理系统进展，第 31 卷。Curran Associates, Inc.，2018年。网址 https://proceedings.neurips.cc/ paper_files/paper/2018/file/e6d8545daa42d5ced125a4bf747b3688-Paper.pdf。

Ralph Allan Bradley 和 Milton E. Terry。不完全区组设计的秩分析。*Biometrika*，39 345，1952年 3 4 324 10 1093 39 3 4 3241010933934324(-): – . doi: . /biomet/ . - . . 网址 https://doi.org/./biomet/.-.。

Murray Campbell、A.Joseph Hoane 和许峰雄。深蓝。人工智能，134(1-2):57–83，2002年1月。ISSN 0004-3702。doi: 10.1016/s0004-3702(01)00129-1。网址 http://dx.doi.org/10.1016/S0004-3702(01) 00129-1。

Yizhe Chi、Wenyi Li、Deyao Hong、Xiaoqiu Wang、Mingju Gao、Kaisen Yang、Bingxiang He、Youjie Zheng、Calvin Xiao 和 Qinhuai Na。Ai4ai-bench：面向递归自我改进的大语言模型智能体算法设计基准测试，2026年。网址 https://arxiv.org/abs/2608.20318。

Anthony Costarelli、Mat Allen、Roman Hauksson、Grace Sodunke、Suhas Hariharan、Carlson Cheng、Wenjie Li、Joshua Clymer 和 Arjun Yadav。GameBench：评估大语言模型智能体的战略推理能力，2024年。网址 https://arxiv.org/abs/2406.06613。

Rémi Coulom. 神兽林格霍姆搜索中的高效选择性与备份算子，第72–83页。Springer Berlin Heidelberg，2007年。ISBN 9783540755388。doi: 10.1007/978-3-540-75538-8_7。网址：http://dx.doi.org/10.1007/ 978-3-540-75538-8_7。

Panayiotis Danassis 和 Naman Goel。人类编码与竞争对手编码：市场驱动的战略规划编程锦标赛，2025年。网址：https://arxiv.org/abs/2511.20613。

Bovard Doerschuk-Tiberi、Yao Yan、Justin Chiu、Hann Wang、Timothy Chung、Martyna Plomecka、John Schultz、Jon Lipovetz、Clayton Drazner、Yuchen Zhuang、Jaimie Hwang、Nate Keating、Riley Jones、Andrew Lee、Oran Kelly、Ian Gemp、Michael Aaron、Laurel Prince、Kate Larson、Jeff Moser、Harrison Jobe、Chad Woodford、Siqi Liu、Andrew Wang、Bo Chang、Christopher D' Mello、Diane Chaleff、Addison Howard、Johnny Yip、Chuck Sugnet、Antonio Gulli、Meghan O' Connell、Will Cukierski、Nenad Tomasev、Dima Yeroshenko、Kinjal Parekh、Roxanne Daniel、Marc Lanctot、Domino Weir、Elsa Dong、Daniel Hennes、Melissa Nalubwama、Robert Fraser、Ryan Trostle、Jun Peng、Tom Mason、Lloyd Hightower、Chiamaka Chukwuka、Yuexiang Zhai、Phoebe Kirk、Yi Su、Yuting Han、Jie Ren、Chris Prichard、Sahand Sharifzadeh、Karim Hakimzadeh、DJ Sterling、Meg Risdal、Kate Olszewska、Ya Xu、Orhan Firat 和 Minmin Chen。Game arena：竞争环境中的策略性大语言模型评估，2026年。网址：https://arxiv.org/abs/2609.31473v1。

Manuel Eberhardinger、James Goodman、Alexander Dockhorn、Diego Perez-Liebana、Raluca D. Gaina、Duygu Çakmak、Setareh Maghsudi 和 Simon Lucas。从代码到游戏：使用大语言模型对游戏程序搜索进行基准测试，2025年。网址 https://arxiv.org/abs/2412.04057。

Lingyue Fu、Xin Ding、Linyue Pan、Yaoming Zhu、Shao Zhang、Lin Qiu、Weiwen Liu、Weinan Zhang、Xuezhi Cao、Xunliang Cai、Jiaxin Ding 和 Yong Yu。CATArena：通过迭代锦标赛评估代码智能体的进化能力，2026年。网址 https://arxiv.org/abs/2510.26852。

Game Agent Coding League . 面向大语言模型的 Game Agent Coding League . 官方基准测试文档，n .d. 网址https://gameagentcodingleague.com/。访问日期：2026年9月28日；未注明发布日期。

Paul Garnier、Jonathan Viquerat 和 Elie Hachem。使用编程智能体进行主动流控制的启发式学习，2026年。网址：https://arxiv.org/abs/2607.11565。

Leon Guertler、Bobby Cheng、Simon Yu、Bo Liu、Leshem Choshen 和 Cheston Tan . TextArena，2025年 . 网址https://arxiv.org/abs/2504.11442。

Pablo Hernandez-Leal, Bilal Kartal, and Matthew E. Taylor. A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 33(6):750–797, October 2019. ISSN 1573-7454. doi: 10.1007/s10458-019-09421-1. URL http://dx.doi.org/10.1007/s10458-019-09421-1.

Lanxiang Hu, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P. Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. lmgame-bench: How good are llms at playing games?, 2025a. URL https://arxiv.org/abs/2505.15146.

Lanxiang Hu, Qiyu Li, Anze Xie, Nan Jiang, Ion Stoica, Haojian Jin, and Hao Zhang. Gamearena: Evaluating llm reasoning through live computer games, 2025b. URL https://arxiv.org/abs/2412.06394.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025c. URL https://arxiv.org/abs/2408.08435.

Haidar Khan, Hisham A. Alyahya, Yazeed Alnumay, M Saiful Bari, and Bülent Yener. Zerosumeval: Scaling llm evaluation with inter-model competition, 2025. URL https://arxiv.org/abs/2504.12562.

Zhiyi Kuang, Ryan Rong, YuCheng Yuan, and Allen Nie. Learning game-playing agents with generative code optimization, 2025. URL https://arxiv.org/abs/2508.19506.

Wenye Lin, Jonathan Roberts, Yunhan Yang, Samuel Albanie, Zongqing Lu, and Kai Han. GAMEBoT: Transparent assessment of LLM reasoning in games. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7656–7682, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.378. URL https://aclanthology.org/2025.acl-long.378/.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models, 2024. URL https://arxiv.org/abs/2310.12931.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. Balrog: Benchmarking agentic llm and vlm reasoning on games, 2025. URL https://arxiv.org/abs/2411.13543.

Wenjun Peng, Xinyu Wang, and Qi Wu. Proxywar: Dynamic assessment of llm code generation in game arenas, 2026. URL https://arxiv.org/abs/2602.04296.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475, December 2023. ISSN 1476-4687. doi: 10.1038/s41586-023-06924-6. URL http://dx.doi.org/10.1038/s41586-023-06924-6.

Saiblo. Saiblo. https://www.saiblo.net/, n.d. Accessed October 9, 2026.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419): 1140–1144, December 2018. ISSN 1095-9203. doi: 10.1126/science.aar6404. URL http://dx.doi.org/10.1126/science.aar6404.

Abhinav Verma, Vijayaraghavan Murali, Rishabh Singh, Pushmeet Kohli, and Swarat Chaudhuri. Programmatically interpretable reinforcement learning. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 5045–5054. PMLR, 10–15 Jul 2018. URL https://proceedings.mlr.press/v80/verma18a.html.

Paul M.B. Vitányi. How incomputable is kolmogorov complexity? *Entropy*, 22(4):408, April 2020. ISSN 1099-4300. doi: 10.3390/e22040408. URL http://dx.doi.org/10.3390/e22040408.

Zhilin Wang, Han Song, Runzhe Zhan, Jusen Du, Jiacheng Chen, Tianle Li, Qingyu Yin, Yulun Wu, Zhennan Shen, Tong Zhu, Yanshu Li, Guanjie Chen, Derek F. Wong, Yafu Li, Yu Cheng, and Yang Yang. Evopolicygym: Evaluating autonomous policy evolution in interactive environments, 2026. URL https://arxiv.org/abs/2607.02440.

Pablo Hernandez-Leal、Bilal Kartal 和 Matthew E. Taylor。多智能体深度强化学习综述与评析。自主智能体与多智能体系统，33(6):750–797，2019年10月。ISSN 1573-7454。doi: 10.1007/s10458-019-09421-1。网址 http://dx.doi.org/10.1007/s10458-019-09421-1。Lanxiang Hu、Mingjia Huo、Yuxuan Zhang、Haoyang Yu、Eric P. Xing、Ion Stoica、Tajana Rosing、Haojian Jin 和 Hao Zhang。LMGame-Bench：大语言模型玩游戏的能力有多强？，2025年a。网址 https://arxiv.org/abs/2505.15146。Lanxiang Hu、Qiyu Li、Anze Xie、Nan Jiang、Ion Stoica、Haojian Jin 和 Hao Zhang。GameArena：通过实时电脑游戏评估大语言模型推理，2025年b。网址 https://arxiv.org/abs/2412.06394。Shengran Hu、Cong Lu 和 Jeff Clune。智能体系统的自动化设计，2025年c。网址 https://arxiv.org/abs/ 2408.08435。Haidar Khan、Hisham A. Alyahya、Yazeed Alnumay、M Saiful Bari 和 Bülent Yener。ZeroSumEval：通过模型间竞争扩展大语言模型评估，2025年。网址 https://arxiv.org/abs/2504.12562。Zhiyi Kuang、Ryan Rong、YuCheng Yuan 和 Allen Nie。通过生成式代码优化学习游戏智能体，2025年。网址 https://arxiv.org/abs/2508.19506。Wenye Lin、Jonathan Roberts、Yunhan Yang、Samuel Albanie、Zongqing Lu 和 Kai Han。GAMEBoT：对游戏中大语言模型推理的透明评估。收录于 Wanxiang Che、Joyce Nabende、Ekaterina Shutova 和 Mohammad Taher Pilehvar 主编的《计算语言学协会第63届年会论文集（第1卷：长篇论文）》，第7656–7682页，奥地利维也纳，2025年7月。计算语言学协会。ISBN 979-8-89176-251-0。doi: 10.18653/v1/2025.acl-long.378。网址 https://aclanthology.org/2025.acl-long.378/。Yecheng Jason Ma、William Liang、Guanzhi Wang、De-An Huang、Osbert Bastani、Dinesh Jayaraman、Yuke Zhu、Linxi Fan 和 Anima Anandkumar。Eureka：通过编程大语言模型实现人类水平的奖励设计，2024年。网址 https://arxiv.org/abs/2310.12931。Alexander Novikov、Ngân V˜u、Marvin Eisenberger、Emilien Dupont、Po-Sen Huang、Adam Zsolt Wagner、Sergey Shirobokov、Borislav Kozlovskii、Francisco J. R. Ruiz、Abbas Mehrabian、M. Pawan Kumar、Abigail See、Swarat Chaudhuri、George Holland、Alex Davies、Sebastian Nowozin、Pushmeet Kohli 和 Matej Balog。Alphaevolve：用于科学与算法发现的编程智能体，2025年。网址 https://arxiv.org/abs/2506.13131。Davide Paglieri、Bartłomiej Cupiał、Samuel Coward、Ulyana Piterbarg、Maciej Wolczyk、Akbir Khan、Eduardo Pignatelli、Łukasz Kuciński、Lerrel Pinto、Rob Fergus、Jakob Nicolaus Foerster、Jack Parker-Holder 和 Tim Rocktäschel。BALROG：以游戏为基准测试智能体式大语言模型与视觉语言模型推理，2025年。网址 https://arxiv.org/abs/2411.13543。Wenjun Peng、Xinyu Wang 和 Qi Wu。ProxyWar：在游戏竞技场中动态评估大语言模型代码生成，2026年。网址 https://arxiv.org/abs/2602.04296。Bernardino Romera-Paredes、Mohammadamin Barekatain、Alexander Novikov、Matej Balog、M. Pawan Kumar、Emilien Dupont、Francisco J. R. Ruiz、Jordan S. Ellenberg、Pengming Wang、Omar Fawzi、Pushmeet Kohli 和 Alhussein Fawzi。通过大语言模型程序搜索实现数学发现。自然，625(7995): 468–475，2023年12月。ISSN 1476-4687。doi: 10.1038/s41586-023-06924-6。网址 http://dx.doi.org/10.1038/ s41586-023-06924-6。Saiblo。Saiblo。https://www.saiblo.net/，无日期。访问日期：2026年10月9日。David Silver、Thomas Hubert、Julian Schrittwieser、Ioannis Antonoglou、Matthew Lai、Arthur Guez、Marc Lanctot、Laurent Sifre、Dharshan Kumaran、Thore Graepel、Timothy Lillicrap、Karen Simonyan 和 戴密斯·哈萨比斯。一种通过自我对弈掌握国际象棋、将棋和围棋的通用强化学习算法。科学，362(6419): 1140–1144，2018年12月。ISSN 1095-9203。doi: 10.1126/science.aar6404。网址 http://dx.doi.org/10.1126/science. aar6404。Abhinav Verma、Vijayaraghavan Murali、Rishabh Singh、Pushmeet Kohli 和 Swarat Chaudhuri。程序化可解释强化学习。收录于 Jennifer Dy 和 Andreas Krause 主编的《第35届国际机器学习大会论文集》，机器学习研究论文集第80卷，第5045–5054页。PMLR，2018年7月10日至15日。网址 https://proceedings.mlr.press/v80/verma18a.html。Paul M.B. Vitányi。柯尔莫哥洛夫复杂度究竟有多不可计算？熵，22(4):408，2020年4月。ISSN 1099-4300。doi: 10.3390/e22040408。网址 http://dx.doi.org/10.3390/e22040408。Zhilin Wang、Han Song、Runzhe Zhan、Jusen Du、Jiacheng Chen、Tianle Li、Qingyu Yin、Yulun Wu、Zhennan Shen、Tong Zhu、Yanshu Li、Guanjie Chen、Derek F. Wong、Yafu Li、Yu Cheng 和 Yang Yang。EvoPolicyGym：在交互式环境中评估自主策略演化，2026年。网址 https://arxiv.org/abs/2607.02440。

Jiayi Weng. Learning beyond gradients. https://trinkle23897.github.io/learning-beyond-gradients/, May 2026. Blog post.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL https://arxiv.org/abs/2404.07972.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL https://arxiv.org/abs/2405.15793.

John Yang, Kilian Lieret, Joyce Yang, Carlos E. Jimenez, Muhtasham Oblokulov, Aryan Siddiqui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Codeclash: Benchmarking goal-oriented software engineering, 2026a. URL https://arxiv.org/abs/2511.00839v2.

Kaisen Yang, Zheng Jiang, Yuzhao Peng, Houde Qian, Boshi Zhang, Youjie Zheng, Shijin Hong, Qingle Liu, Ruoyu Han, Bohan Lyu, Bingxiang He, Eren Cai, Calvin Xiao, and Qinhuai Na. Scalable behaviour cloning on browser using via skill distillation, 2026b. URL https://arxiv.org/abs/2606.32014.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation, 2024. URL https://arxiv.org/abs/2310.02304.

A The evaluated games

The suite shares an evaluation interface, but its policies solve different decision problems. The descriptions below refer to the public rule and SDK bundles used in the experiments; Table 1 lists pool sizes.

Pacman. *Pacman* is a separate, symmetric competitive maze game. Players collect ordinary and `high-value beans`, avoid `monsters`, and place `mines` that affect movement choices. Collection, collision avoidance, and disruption of the opponent jointly determine the score. Its rules and opponent pool are distinct from *Rollman*'s asymmetric contest.

SnakeGo. *SnakeGo* is a territory game in which `snakes` move, collect items, claim `cells`, and can split or use special actions. Local collision avoidance interacts with longer-horizon territory growth, making a safe immediate move potentially costly at settlement.

Rollman. *Rollman* is an asymmetric maze game. One side collects `beans` and uses temporary skills; the other coordinates `ghosts` to intercept it. The roles differ in both action space and objective, making movement planning and coordinated pursuit distinct policy problems.

MoneCraft. *MoneCraft* is a mining and resource-control game on a symmetric map. Players capture `mines` that generate `currency` and can teleport, place `mines`, or purchase protection. The policy trades movement and disruption against sustained resource production; end-game `currency` and `mine ownership` determine the result under the referee's rules.

AntWar. *AntWar* combines tower defence with economic management. Players build and upgrade `towers`, improve their `bases`, and time `super-weapons`, while `ants` follow environment-driven paths. The policy must balance present defence with spending that changes future pressure.

LostSpace. *LostSpace* is a multiplayer survival-and-escape game on a layered map that contracts over time. Players collect `supplies` and `keys`, use `tools`, fight or evade rivals, and seek escape. Ranked outcomes create trade-offs between immediate combat, survival, and the timing of an escape attempt.

AquaWar. *AquaWar* is a hidden-identity tactical game. Players select `fish`, make assertions about hidden opponent identities, and choose attacks or type-specific skills. Roster design, inference, and information-sensitive timing all affect the outcome.

Jiayi Weng. 超越梯度的学习. https://trinkle23897.github.io/learning-beyond-gradients/，2026年5月。博客文章。

Tianbao Xie、Danyang Zhang、Jixuan Chen、Xiaochuan Li、Siheng Zhao、Ruisheng Cao、Toh Jing Hua、Zhoujun Cheng、Dongchan Shin、Fangyu Lei、Yitao Liu、Yiheng Xu、Shuyan Zhou、Silvio Savarese、Caiming Xiong、Victor Zhong 和 Tao Yu。Osworld：真实计算机环境中面向开放式任务的多模态智能体基准测试，2024年。网址：https://arxiv.org/abs/2404.07972。

John Yang、Carlos E. Jimenez、Alexander Wettig、Kilian Lieret、Shunyu Yao、Karthik Narasimhan 和 Ofir Press。Swe-agent：智能体-计算机接口实现自动化软件工程，2024年。网址：https://arxiv.org/abs/ 2405.15793。

John Yang、Kilian Lieret、Joyce Yang、Carlos E. Jimenez、Muhtasham Oblokulov、Aryan Siddiqui、Ofir Press、Ludwig Schmidt 和 Diyi Yang。CodeClash：目标导向的软件工程基准测试，2026a。网址：https://arxiv.org/abs/2511.00839v2。

Kaisen Yang、Zheng Jiang、Yuzhao Peng、Houde Qian、Boshi Zhang、Youjie Zheng、Shijin Hong、Qingle Liu、Ruoyu Han、Bohan Lyu、Bingxiang He、Eren Cai、Calvin Xiao 和 Qinhuai Na。通过技能蒸馏实现可扩展的浏览器使用行为克隆，2026年b。网址：https://arxiv.org/abs/2606.32014。

Eric Zelikman、Eliana Lorch、Lester Mackey 和 Adam Tauman Kalai。自学优化器（stop）：递归自我改进的代码生成，2024年。网址：https://arxiv.org/abs/2310.02304。

A 评估的游戏

这组游戏共用一个评估接口，但各自的策略解决的是不同的决策问题。以下描述依据实验中使用的公开规则和软件开发工具包；表 1 列出了池大小。

吃豆人。吃豆人是一款独立的对称竞技迷宫游戏。玩家收集普通豆子和高价值豆子、躲避怪物，并布置会影响移动选择的地雷。收集、避碰和干扰对手共同决定得分。其规则和对手池均不同于 *Rollman* 的非对称对抗。

SnakeGo。*SnakeGo* 是一款领地游戏，蛇在其中移动、收集物品、占领网格单元，还可以分裂或使用特殊动作。局部避碰与更长时间尺度上的领地扩张相互影响，因此，眼下安全的一步可能在结算时付出高昂代价。

Rollman。*Rollman* 是一款非对称迷宫游戏。一方收集豆子并使用临时技能；另一方则协调幽灵进行拦截。双方角色的动作空间和目标均不相同，因此移动规划与协同追捕构成了不同的策略问题。

MoneCraft。*MoneCraft* 是一款在对称地图上进行的采矿与资源控制游戏。玩家占领能够产生货币的矿场，并可进行传送、布设地雷或购买防护。策略需要在移动和干扰对手与持续的资源生产之间进行权衡；终局货币和矿场所有权依据裁判规则决定结果。

AntWar。*AntWar* 将塔防与经济管理相结合。玩家建造和升级防御塔、改进基地，并把握使用超级武器的时机，而蚂蚁则沿着环境驱动路径行进。策略必须在当前防御与改变未来压力的开支之间取得平衡。

LostSpace。*LostSpace* 是一款多人生存逃脱游戏，在随时间推移而收缩的分层地图上进行。玩家收集补给和钥匙、使用工具、与对手战斗或躲避对手，并寻求逃脱。排名结果使玩家需要在即时战斗、生存和逃脱尝试的时机之间进行权衡。

AquaWar。*AquaWar* 是一款隐藏身份战术游戏。玩家选择鱼类，对对手隐藏身份作出断言，并选择攻击或类型专属技能。阵容设计、推断和信息敏感的时机选择都会影响结果。

Generals. *Generals* combines territorial expansion, army movement, neutral assets, production, technology, and tactical abilities. An effective policy coordinates economic development with the timing and direction of attacks rather than optimizing either in isolation.

Dorado. *Dorado* combines resource competition with `hero` and army development. Players choose `heroes`, produce and command `units`, mine neutral resources, and invest in `upgrades`. The policy coordinates movement, combat, harvesting, and abilities to threaten opposing `bases` while maintaining the resources needed to continue production.

Miracle. *Miracle* is a hex-grid strategy game with heterogeneous `units` and an initial drafting stage. A player selects `creatures` and an `artifact`, then summons, moves, attacks, and uses abilities to destroy the opponent's `miracle`. Resource limits and unit interactions make tactical coordination central to the policy.

LOTA. *LOTA* is a lane-based combat game. Each player chooses a `hero` and controls movement, attacks, abilities, skill development, and `revival` while `units` advance along fixed `lanes`. Victory depends on destroying the opposing `base` or the referee's settlement criteria at the turn limit. Effective control couples immediate fights with sustained pressure on defensive structures.

AntWar2. *AntWar2* extends tower-defence strategy with `ordered operation lists` whose accepted prefix changes subsequent decisions. The agent must coordinate `tower placement`, economic development, and weapon timing while respecting sequence-dependent legality.

B Arena resources and evaluation details

Public resources and opponents. We freeze the versioned opponent programs, their ratings, and the game resources across runs. Some entrants submitted several related versions, so pool sizes count programs, not independent participants. The workspace includes rules, SDKs, replay documentation, a leaderboard, and one reference program (the rank-40 entry). The agent cannot access the other opponents' source code or evaluator internals.

Budget accounting. A small-match request tests up to 8 opponents and costs one unit per opponent. A full-pool evaluation costs one large unit. These budgets constrain feedback access; source edits, model calls, tokens, and wall-clock time have no separate limits. The starter policy's initial evaluation is free and falls outside the charged full-pool sequence.

B.1 AI agent workflow

Each model–game run uses one persistent session with a writable workspace. Opus5.5 runs in Claude Code; the other models run in Codex. The agent's goal is to improve competitive performance. It chooses when to read rules, edit code, test selected opponents, and submit a full-pool evaluation. Small matches return replays; full-pool evaluations return Elo, rank, and per-opponent outcomes. Context compaction preserves the session but shortens its conversation history, so the agent keeps notes to retain experience. We report the run's highest officially evaluated Elo, including the common initialization baseline, and its corresponding rank. The controller retains the policy snapshot attaining that score.

B.2 Elo and rank

Each full-pool evaluation estimates the Elo $\widehat{r}(P)$ of policy snapshot P from its match outcomes using a fixed-pool Bradley–Terry model (Bradley and Terry, 1952). Let H denote the frozen opponent pool and r_j the fixed rating of opponent j. The candidate's score in match m against opponent $j(m)$ is $s_m \in \{0, \frac{1}{2}, 1\}$ (loss, draw, win). The expected score at rating r is

$$p(r, r_j) = \frac{1}{1 + 10^{(r_j - r)/400}},$$

Generals。*Generals* 融合了领土扩张、军队移动、中立资产、生产、科技和战术技能。有效的策略会协调经济发展与攻击的时机和方向，而不是孤立地优化其中任何一方。

Dorado。*Dorado* 将资源争夺与英雄和军队发展相结合。玩家选择英雄、生产并指挥单位、采集中立资源，并投入资源进行升级。策略需要协调移动、战斗、采集和技能，在威胁敌方基地的同时，维持持续生产所需的资源。

Miracle。*Miracle* 是一款包含异构单位和初始选秀阶段的六边形网格策略游戏。玩家选择生物和神器，然后通过召唤、移动、攻击和使用技能来摧毁对手的奇迹。资源限制和单位交互使战术协同成为策略的核心。

LOTA。*LOTA* 是一款分路战斗游戏。每位玩家选择一名英雄，控制其移动、攻击、技能、技能发展和复活，同时各单位沿固定兵线推进。胜负取决于能否摧毁敌方基地，或达到回合上限时裁判的结算标准。有效的控制需要将眼前的战斗与对防御建筑的持续施压相结合。

AntWar2。*AntWar2* 通过引入有序操作列表扩展了塔防策略，其已接受前缀会改变后续决策。智能体必须在遵守序列依赖合法性的同时，协调防御塔布置、经济发展和武器使用时机。

B 竞技场资源与评估细节

公共资源与对手。我们在各次运行中固定对手程序的版本及其等级分，并保持游戏资源不变。部分参赛者提交了多个相关版本，因此池大小按程序数量而非独立参赛者人数计算。工作区包含规则、软件开发工具包、回放文档、排行榜和一个参考程序（排名第40的参赛程序）。智能体无法访问其他对手的源代码或评估器内部机制。

预算核算。一次小规模对战请求最多可测试8个对手，每个对手消耗一个预算单位。一次全对手池评估消耗一个大额预算单位。这些预算限制反馈获取；源代码修改、模型调用、词元和实际耗时均无单独限制。初始策略的首次评估免费，不计入收费的全对手池评估序列。

B.1 人工智能智能体工作流程

每次模型—游戏运行均使用一个配有可写工作区的持久会话。Opus5.5 在 Claude Code 中运行，其他模型在 Codex 中运行。智能体的目标是提升竞技表现。它自行决定何时阅读规则、编辑代码、测试选定对手，以及提交全对手池评估。小规模对局返回对局回放；全池评估返回埃洛等级分、排名和各对手对战结果。上下文压缩会保留会话，但会缩短其对话历史，因此智能体通过记录笔记来保留经验。我们报告该次运行经正式评估所得的最高埃洛等级分及其对应排名，其中也包括共同的初始化基线。控制器会保留取得该分数的策略快照。

B.2 埃洛等级分与排名

每次全对手池评估都会估计埃洛等级分 $\widehat{r}(P)$，该等级分对应策略快照 P，并根据其对战结果使用固定对手池Bradley–Terry模型进行估计（Bradley 和 Terry，1952年）。令 H 表示冻结的对手池，r_j 表示对手 j 的固定等级分。候选策略在对局 m 中对阵对手 j（m）时的得分为 $s_m \in \{0, \frac{1}{2}, 1\}$（失败、平局、获胜）。等级分为 r 时的预期得分为 –

$$p(r, r_j) = \frac{1}{1 + 10^{(r_j - r)/400}},$$

and the estimate $\hat{r}$ solves

$$\sum_m p(\hat{r}, r_{j(m)}) + p(\hat{r}, \bar{r}_H) = \sum_m s_m + \tfrac{1}{2}, \qquad \bar{r}_H = \frac{1}{|H|}\sum_{j\in H} r_j.$$

The extra term adds one virtual draw against the complete pool's mean rating, keeping $\hat{r}$ finite for perfect win or loss records. Opponent ratings remain fixed across snapshots and runs. The candidate's rank is $1 + |\{j \in H : r_j > \hat{r}\}|$. We report each game's native Elo and rank and summarize rank-1 coverage with gold medals.

B.3 Gold medals

Gold medals count rank-1 finishes against the frozen human ladders. For configuration m and game g, let $q_{m,g}$ denote its reported rank. Across $G = 12$ games, we compute

$$\mathrm{Gold}(m) = \sum_{g=1}^{G} \mathbf{1}\{q_{m,g} = 1\}.$$

A gold medal denotes first place under this rating protocol; it does not establish a direct head-to-head win against the human leader or optimal play.

The pool ratings and rank-1 thresholds remain fixed when we evaluate additional models. Gold medals summarize performance against the archived pools. Full-pool evaluations provide feedback during optimization, so the reported results do not measure generalization to unseen opponents. The committed summary records pin the rating inputs and calculations; they do not independently validate the original human-rating calibration.

B.4 Rule complexity: AST and RA

We compare rule-description sizes by rewriting each game's rules in a shared formal pseudocode language. Its domain-neutral vocabulary includes top-level declarations (`constant`, `enum`, `entity`, `action`, `observation`, `setup`, `rule`, `terminal`), fifteen statement types, and expressions for arithmetic, comparisons, and function calls. Statement types include `let`, `require`, `when`, `for`, `update`, `create`, `delete`, `reveal`, and `return`. Set membership, distance, and probability are primitives. We explicitly define every game-specific operation, such as splitting a snake or casting a fish skill. Each description covers players, information, state, initialization, randomness, legal actions, transitions, and termination. We exclude communication protocols, rendering, logging, and platform-only timeouts.

AST. The primary measure is the node count of the description's canonical abstract syntax tree,

$$C_{\mathrm{AST}}(G) = N_{\mathrm{structure}}(d_G) + N_{\mathrm{expression}}(d_G),$$

where d_G describes game G. $N_{\mathrm{structure}}$ counts declaration, block, and statement nodes; $N_{\mathrm{expression}}$ counts calls, names, attributes, literals, collections, subscripts, comparisons, and operators. We exclude parser-only wrapper and context nodes and count each named rule once at its definition and once at every call site.

RA. The secondary measure, rule atoms (RA), counts independently changeable rule propositions. Each declaration member or rule statement expresses exactly one such proposition; two effects written in one sentence still count as two. RA is the size of the disjoint union of seven proposition classes: state, action, observation, setup, condition, transition, and outcome.

Both measures are invariant to identifier renaming, comments, and formatting. They depend on the chosen language and measure the size of the rule description, not the state-space size or the difficulty of optimal play.

B.5 Reading the trajectory plots

The single-run plots present three complementary views of policy development. Panel (a) plots each submitted snapshot's full-pool Elo, with aligned rank annotations and frozen-ladder reference levels. These curves can regress; the row heading separately reports the retained champion.

且估计值 $\hat{r}$ 满足

$$\sum_m p(\hat{r}, r_{j(m)}) + p(\hat{r}, \bar{r}_H) = \sum_m s_m + \tfrac{1}{2}, \qquad \bar{r}_H = \frac{1}{|H|}\sum_{j\in H} r_j.$$

附加项相当于增加一场与完整池平均等级分对手的虚拟平局，使$\hat{r}$ 在全胜或全负的胜负记录下仍为有限值。对手等级分在不同快照和运行中保持固定。候选者的排名为$1 + |\{j \in H :\ r_j > \hat{r}\}|$。我们报告每款游戏的原生Elo评分和排名，并用金牌汇总榜首覆盖率。

B.3 金牌

金牌用于统计在冻结的人类排行榜上取得榜首的次数。对于配置 m 和游戏 g，令$q_{m,g}$ 表示其报告的排名。对于 $G = 12$ 款游戏，我们计算

$$\mathrm{Gold}(m) = \sum_{g=1}^{G} \mathbf{1}\{q_{m,g} = 1\}.$$

金牌表示在此评分规程下获得第一名；它并不能证明直接对战胜过人类领先者，也不能证明达到了最优博弈。

在评估其他模型时，对手池等级分和榜首阈值保持固定。金牌概括了模型相对于归档对手池的表现。全池评估在优化期间提供反馈，因此报告的结果并不衡量对未见对手的泛化能力。已提交的汇总记录固定了评分输入和计算过程；它们并不能独立验证最初的人类等级分校准。

B.4 规则复杂度：抽象语法树和规则原子

我们将各游戏的规则改写为一种共用的形式化伪代码语言，以比较规则描述规模。该语言的领域中立词汇包括顶层声明（常量、枚举、实体、动作、观测、初始设置、规则、终止）、十五种语句类型，以及用于算术、比较和函数调用的表达式。语句类型包括绑定、要求、条件触发、循环、更新、创建、删除、揭示和返回。集合隶属关系、距离和概率均为原语。我们明确定义每一种游戏特定操作，例如分裂蛇或施放鱼类技能。每份描述均涵盖玩家、信息、状态、初始化、随机性、合法动作、状态转移和终止。我们不纳入通信协议、渲染、日志记录和仅限平台的超时机制。

AST. The 主要度量指标是描述的规范抽象语法树的节点数，

$$C_{\mathrm{AST}}(G) = N_{\mathrm{structure}}(d_G) + N_{\mathrm{expression}}(d_G),$$

其中， d_G 描述游戏 G。 $N_{\mathrm{structure}}$ 统计声明、块和语句节点； $N_{\mathrm{expression}}$ 统计调用、名称、属性、字面量、集合、下标、比较和运算符。我们排除仅供解析器使用的包装节点和上下文节点，并对每条命名规则在其定义处计数一次，在每个调用位置也计数一次。

规则原子。次要度量指标为规则原子（RA），用于统计可独立更改的规则命题。每个声明成员或规则语句恰好表达一个这样的命题；即使两个效果写在同一个句子中，也仍计为两个命题。规则原子数量是七种命题类别的不交并的大小：状态、动作、观测、初始设置、条件、转移和结果。

这两种度量均不受标识符重命名、注释和格式设置的影响。它们取决于所选语言，衡量的是规则描述的大小，而非状态空间大小或最优对弈的难度。

B.5 解读轨迹图

单次运行图从三个互补的视角展示策略开发过程。面板 (a) 绘制了每个提交快照的全池 Elo 评分，并配有对齐的排名标注和冻结天梯参考水平。这些曲线可能回落；行标题则单独报告保留的冠军。

Panel (b) shows interaction order on 144 equally spaced budget slots (128 small-match units and 16 full-pool evaluations). An opponent test or full-pool evaluation occupies one slot. The continued runs use 432 slots (384 small and 48 full). Tests in the same batch share the midpoint of its occupied slots. Small dots give selected opponents' ladder ranks: green denotes wins, magenta losses, and grey draws. Large diamonds give the learner's full-pool rank, with rank 1 at the top. In *Dorado*, 128 opponent tests comprise 16 batches and 256 role outcomes.

Panel (c) reports tokens in millions at each full-pool evaluation. Lines show cumulative use; bars count tokens since the preceding evaluation, with the first bar starting at the run's beginning. Continuation plots annotate ranks at every third full-pool evaluation and at local Elo peaks.

The paired replay-source plots use blue circles for own-policy replays and purple diamonds for other-policy replays. Their panel (a) shows both Elo curves and rank rows; dashed levels refer only to the other-policy run's first evaluated policy and retained champion. Panel (b) shows cumulative tokens as lines and between-evaluation increments as paired bars on the right axis. The ablation bars label retained-champion Elo and rank; colours distinguish conditions, and panels within each figure share a zero-based Elo axis.

B.6 Complete Policy Evolution Trajectories

These plots show the complete trajectories of the twelve GLM-5.3 runs from Section 4.2.

面板 (b) 在 144 个等间距的预算槽位上展示交互顺序（128 个小规模对局单元和 16 次全对手池评估）。一次对手测试或全对手池评估占用一个槽位。延续运行使用 432 个槽位（384 个用于小规模对局，48 个用于完整池评估）。同一批次中的测试共用该批次所占槽位的中点。小圆点表示所选对手的天梯排名：绿色表示胜局，品红色表示负局，灰色表示平局。大菱形表示学习者的全池排名，第 1 名位于顶部。在 *Dorado* 中，128 次对手测试包含 16 个批次和 256 个角色结果。

面板 (c) 报告每次全对手池评估时的词元数，单位为百万。折线表示累计用量；条形表示自上一次评估以来的词元用量，其中第一个条形从运行开始时计起。续跑图在每隔三次全对手池评估处以及局部Elo峰值处标注排名。

成对的回放来源图使用蓝色圆点表示自身策略回放，使用紫色菱形表示其他策略回放。其面板 (a) 同时展示Elo曲线和排名行；虚线水平线仅对应其他策略运行中的首个已评估策略和保留的冠军。面板 (b) 以折线表示累计词元数，并以对应右侧坐标轴的成对条形表示两次评估之间的增量。消融实验条形标注保留冠军的Elo评分和排名；颜色用于区分条件，同一图中的各面板共用以零为起点的Elo坐标轴。

B.6 完整策略演化轨迹

这些图展示了第 4.2 节中十二次 GLM-5.3 运行的完整轨迹。

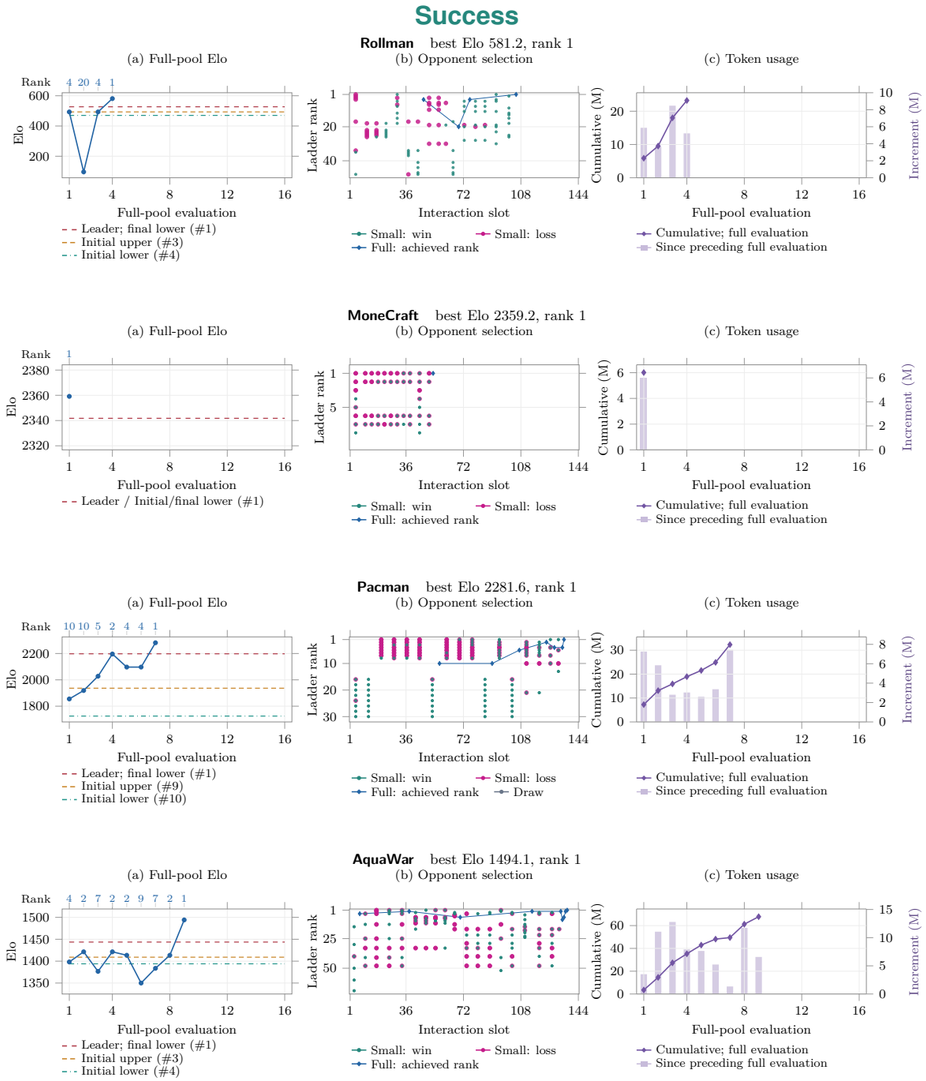

Figure 11 Success. Full-pool Elo and ranks, interaction order, and token use for runs that reach rank 1 and stop early. Plot conventions appear in Appendix B.5.

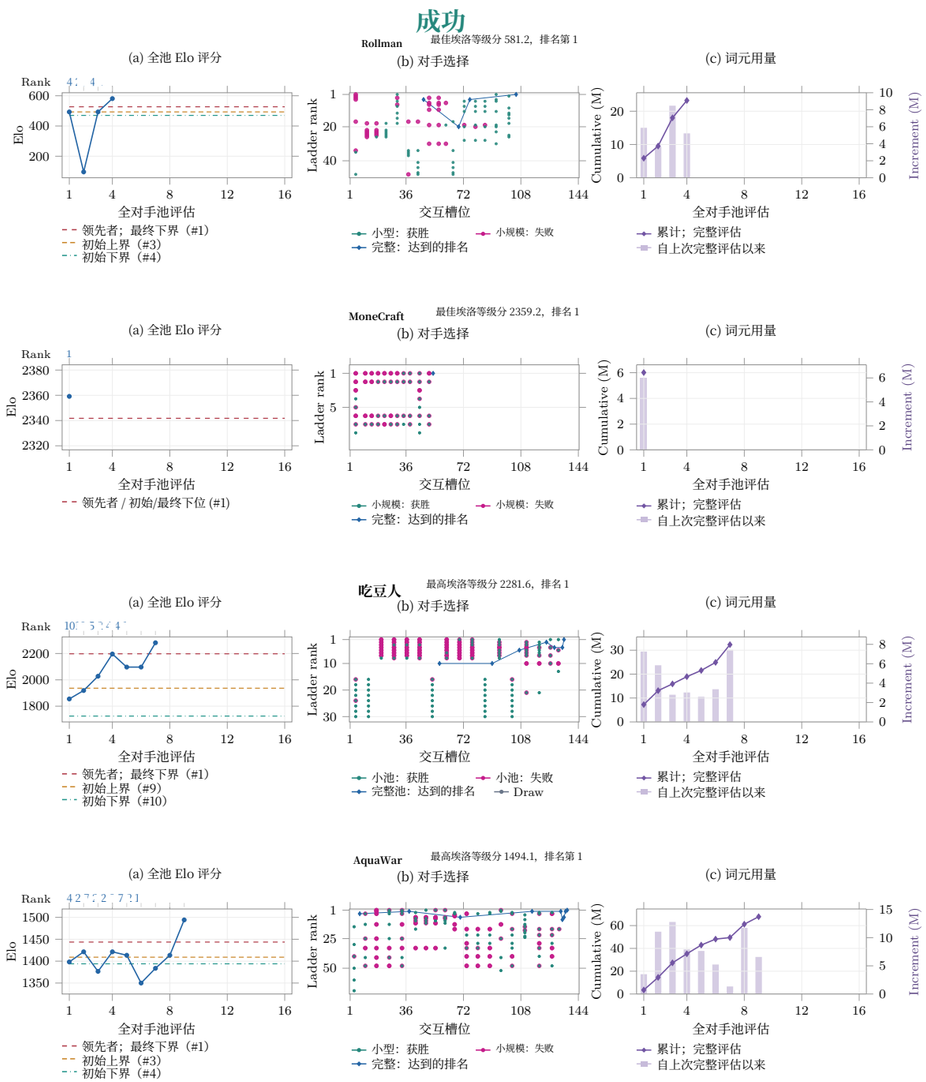

图 11 成功。 达到排名第 1 并停止的运行的全池 Elo 评分与排名、交互顺序及词元使用量 p
提前。绘图约定 见附录 B.5。

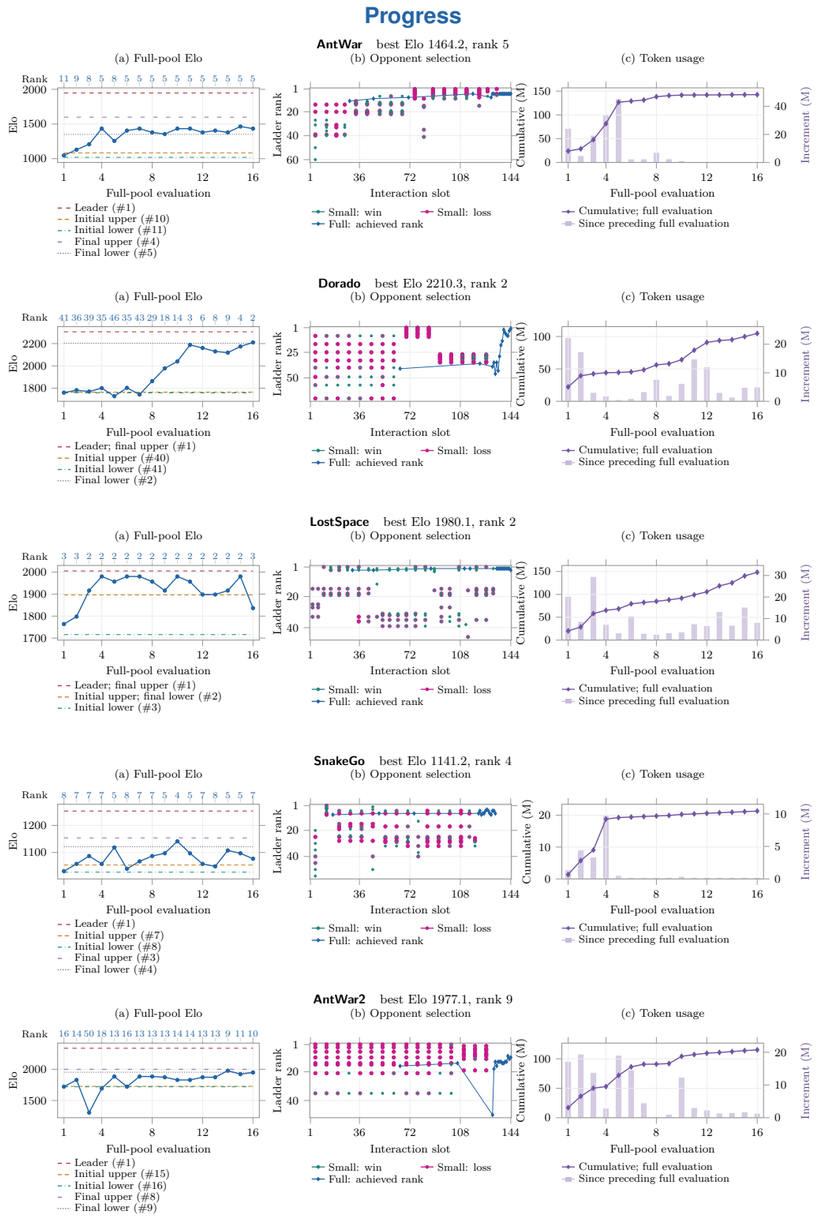

Figure 12 Progress. Full-pool Elo and ranks, interaction order, and token use for games with fluctuating improvement. Plot conventions appear in Appendix B.5.

进展

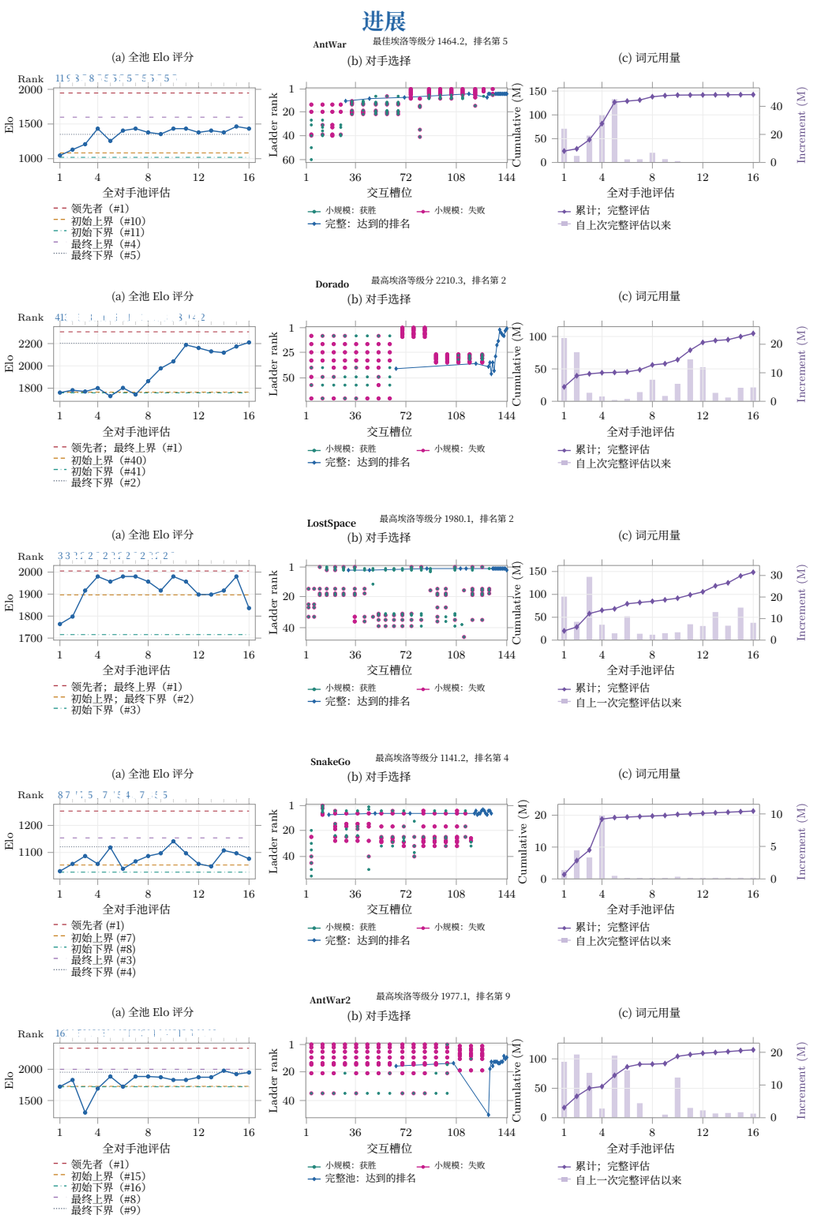

图 12 进展。 改进幅度波动的游戏的全池 Elo 评分及排名、交互顺序和词元使用量 t. 绘图惯例见见附录 B.5。

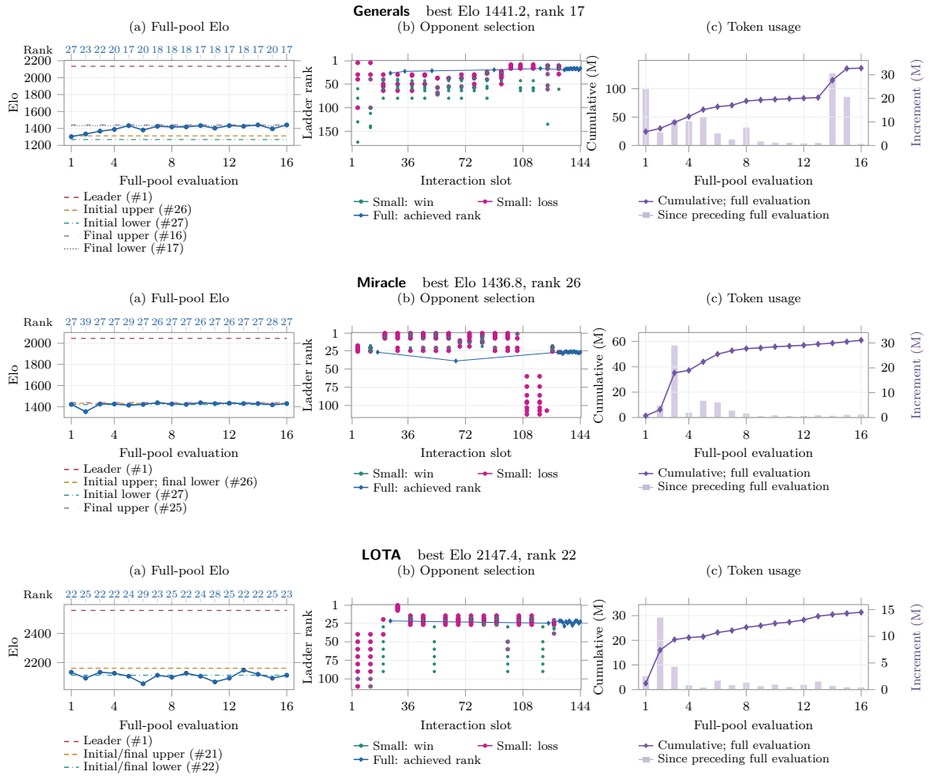

Figure 13 Stagnation. Full-pool Elo and ranks, interaction order, and token use for games that plateau after initial adaptation. Plot conventions appear in Appendix B.5.

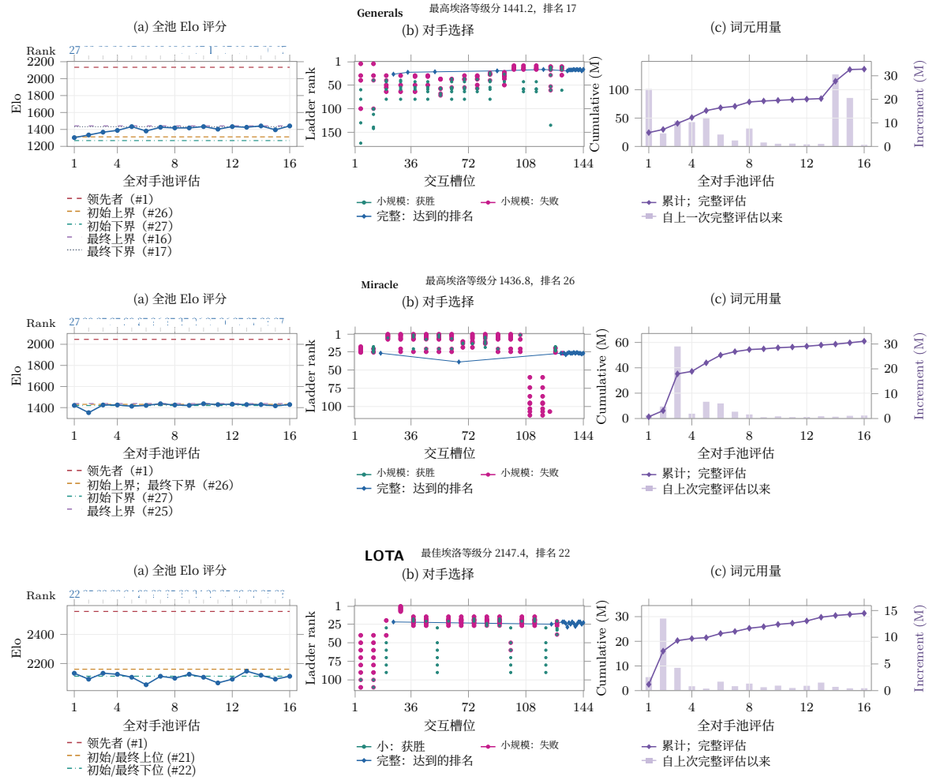

图 13 停滞。初始适应后进入平台期的游戏的全池 Elo 评分与排名、交互顺序及词元使用量。绘图约定见附录 B.5。

C Detailed ablation results

Tables 6–8 list the Elo and rank behind Figures 6–8.

Game	Model	Ladder	Random	Top-4
Pacman	2096.8 / 4	2127.5 / 3	2014.4 / 6	1897.0 / 10
AntWar	1180.5 / 8	1311.7 / 6	1154.4 / 9	1056.2 / 11
Miracle	1482.4 / 20	1448.8 / 24	1457.0 / 24	1406.0 / 31

Table 6 Opponent selection ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

Game	Dense	Binary
Pacman	2236.1 / 1	1710.1 / 11
AntWar	1118.7 / 9	880.6 / 22
Miracle	1528.2 / 15	1461.1 / 23

Table 7 Replay feedback ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

Game	1	2	4	8
Pacman	2040.6 / 5	2196.3 / 2	2703.9 / 1	2160.4 / 2
AntWar	1130.2 / 9	1433.4 / 5	1208.7 / 8	1118.7 / 9
Miracle	1338.7 / 45	1491.2 / 19	1518.7 / 16	1465.3 / 23

Table 8 Small-match batch size ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

D Token consumption

The main text summarizes retained-champion performance. Under the same small-match and full-pool limits, the models consume different numbers of tokens. Table 9 reports the token use of every main-table run.

Equal interaction budgets permit different token expenditure. Total use varies by more than fivefold: GPT6-sol consumes 260M tokens, between 16M and 30M per game, whereas Qwen 3.8 consumes 1,454M. Qwen 3.8 uses 1.6 times GLM-5.3's tokens but earns three gold medals versus four. GPT6-sol also earns four with less than a third of GLM-5.3's tokens. The leading configuration, Opus5.5 with Claude Code, consumes 737M.

Token use also varies within a configuration. Kimi K3 consumes 1.5M on *AquaWar* and 89M on *Generals*. The feedback ceilings therefore do not impose equal computational expenditure; we report token use alongside competitive performance.

Game	Opus5.5	GPT6-sol	GLM-5.3	Kimi K3	DeepSeek V4 Pro	Qwen 3.8	LongCat 2.0
Rollman	48.75	25.28	23.82	5.50	6.11	14.68	18.35
Pacman	22.16	16.14	33.14	40.17	91.95	163.12	17.52
AntWar	74.19	16.08	142.94	54.59	102.80	284.29	37.56
AquaWar	76.95	26.37	68.68	1.49	60.11	13.92	20.65
Generals	75.44	19.66	137.23	88.88	172.70	60.95	22.30
LostSpace	65.37	21.58	149.86	72.03	141.51	190.16	100.95
Miracle	89.29	19.15	61.99	34.78	47.89	107.55	49.05
Dorado	44.58	23.02	105.63	60.72	67.01	253.02	10.14
MoneCraft	56.16	22.10	6.75	29.97	67.54	48.72	31.18
LOTA	76.41	17.36	32.55	30.74	34.04	112.36	21.92
SnakeGo	48.53	23.53	22.49	59.75	29.79	78.77	27.35
AntWar2	58.78	30.07	115.70	71.08	44.08	126.53	67.20
Total	736.61	260.34	900.78	549.70	865.53	1454.07	424.17

Table 9 Token use of the main-table runs (millions, input plus output; we count cached input once within input). All runs share the 128/16 interaction budget.

C 详细消融实验结果

表 6–8 列出了以下各图所对应的埃洛等级分和排名：图 6–8。

Game	模型	阶梯式选择	随机	前四名
吃豆人	2096.8 / 4	2127.5 / 3	2014.4 / 6	1897.0 / 10
AntWar	1180.5 / 8	1311.7 / 6	1154.4 / 9	1056.2 / 11
Miracle	1482.4 / 20	1448.8 / 24	1457.0 / 24	1406.0 / 31

表 6 使用 GLM-5.3 进行的对手选择消融实验。每个单元格表示 128/16 预算下的埃洛等级分 / 排名 t.

Game	稠密	二元
吃豆人	2236.1 / 1	1710.1 / 11
AntWar	1118.7 / 9	880.6 / 22
Miracle	1528.2 / 15	1461.1 / 23

表 7 使用 GLM-5.3 进行的回放反馈消融。每个单元格表示在 128/16 预算下的埃洛等级分 / 排名。

Game	1	2	4	8
吃豆人	2040.6 / 5	2196.3 / 2	2703.9 / 1	2160.4 / 2
AntWar	1130.2 / 9	1433.4 / 5	1208.7 / 8	1118.7 / 9
Miracle	1338.7 / 45	1491.2 / 19	1518.7 / 16	1465.3 / 23

表 8 小规模　对局批大小消融实验，使用 GLM-5.3。每个单元格表示在 128/16 预算下的埃洛等级分 / 排名。

D 词元消耗

正文总结了保留冠军的表现。在相同的小规模对局和完整池限制下，各模型消耗的词元数量不同。表9 报告了主表中每次运行的词元使用量。

相同的交互预算允许不同的词元消耗量。总消耗量相差超过五倍：GPT6-sol 消耗 2.6 亿词元，每个游戏消耗 1600 万至 3000 万词元，而通义千问 3.8 则消耗 14.54 亿词元。通义千问 3.8 的词元消耗量是 GLM-5.3 的 1.6 倍，但仅获得三枚金牌，而后者获得四枚。GPT6-sol 同样获得四枚金牌，词元消耗量却不到 GLM-5.3 的三分之一。排名第一的配置为 Opus5.5 搭配 Claude Code，消耗 7.37 亿词元。

同一配置下的词元使用量也有所不同。Kimi K3 在 *AquaWar* 上消耗 150 万词元，在 *Generals*上则消耗 8900 万词元。因此，反馈上限并不意味着相同的计算开销；我们在报告竞技表现的同时，也报告词元使用量。

Game	Opus5.5	GPT6-sol GLM-5.3	Kimi K3	DeepSeek V4 Pro	通义千问 3.8	LongCat 2.0
Rollman	48.75	25.28	23.82	5.50	6.11	14.68	18.35
吃豆人	22.16	16.14	33.14	40.17	91.95	163.12	17.52
AntWar	74.19	16.08	142.94	54.59	102.80	284.29	37.56
AquaWar	76.95	26.37	68.68	1.49	60.11	13.92	20.65
Generals	75.44	19.66	137.23	88.88	172.70	60.95	22.30
LostSpace	65.37	21.58	149.86	72.03	141.51	190.16	100.95
Miracle	89.29	19.15	61.99	34.78	47.89	107.55	49.05
Dorado	44.58	23.02	105.63	60.72	67.01	253.02	10.14
MoneCraft	56.16	22.10	6.75	29.97	67.54	48.72	31.18
LOTA	76.41	17.36	32.55	30.74	34.04	112.36	21.92
SnakeGo	48.53	23.53	22.49	59.75	29.79	78.77	27.35
AntWar2	58.78	30.07	115.70	71.08	44.08	126.53	67.20
总计	736.61	260.34	900.78	549.70	865.53	1454.07	424.17

表 9 主表各次运行的词元使用量（单位：百万，输入与输出之和；缓存输入在输入中计入一次）。所有运行均采用 128/16 的交互预算。

E Algorithm families in the champion policies

Because the agent acts entirely in code space, the final policies also show which algorithms each model chooses to write. We classify the code of all 84 retained champions into six families:

- **Rule-based control:** hand-written `if`–`else` priorities, state machines, and fixed openings or scripts.
- **Heuristic scoring:** hand-weighted features that score candidate actions or targets, followed by a greedy choice.
- **Graph search:** BFS, DFS, Dijkstra, A*, distance maps, flood fill, and territory partitions.
- **Lookahead search:** search over a forward model, including simulated rollouts, beam search, minimax, expectimax, and MCTS.
- **Combinatorial optimization:** assignment and matching, dynamic programming, and knapsack-style allocation.
- **Inference and randomization:** belief tracking over hidden information, opponent modelling, and mixed or randomized choices.

We analyse source files in each champion's strategy directory, excluding byte-identical SDK files and paths matching version, backup, debugging, test, or vendor filters. We assign each function a family based on its structure and identifiers. For functions longer than 40 lines, we classify 20-line windows separately. We exclude I/O, parsing, and protocol code as infrastructure. A family's share is its fraction of the remaining non-comment lines. Table 10 reports each model–game pair and equally weighted means across models and games.

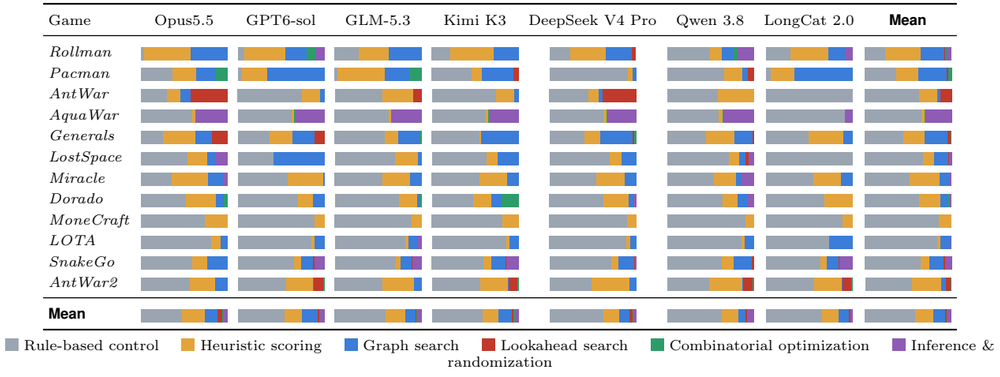

Table 10 Algorithm families in the champion policies. Each bar splits the decision code of one retained champion by algorithm family, weighted by non-comment lines of code. We exclude infrastructure (I/O, parsing, protocol). Row and column means weight each policy equally.

Retained champions rely mainly on rules and heuristic scoring. Averaging the 84 champion-level shares equally, these families account for 79%, graph search for 13%, and the other three families for 8%. Lookahead search exceeds 10% in only 7 of the 84 champions, all on *AntWar*, *AntWar2*, and *Generals*. These endpoint classifications describe policy composition; they do not identify which code changes caused improvements during development.

Algorithm shares differ across games. Graph search accounts for 33% on *Pacman* and 28% on *Rollman*; inference accounts for 30% on *AquaWar*. *MoneCraft* and *LOTA* consist almost entirely of rules. Across models, Opus5.5 has the lowest rule share (47%) and the highest scoring (27%) and lookahead shares (5%); GPT6-sol has the highest graph-search share (18%). The other models range from 58% to 65% rules.

Within each game, we correlate non-rule share (1− rule-based fraction) with Elo across the seven configurations, using average ranks for ties. The mean Spearman correlation across 12 games is 0.35, with variation across

E 冠军策略中的算法族

由于智能体完全在代码空间中行动，最终策略也揭示了各模型选择编写哪些算法。我们将全部 84 个保留的冠军的代码分为六类：

- **基于规则的控制：**手工编写的if–else优先级规则、状态机，以及固定开局或脚本。
- **启发式评分：**使用手动加权特征对候选动作或目标进行评分，随后进行贪心选择。
- **图搜索：**广度优先搜索、深度优先搜索、迪杰斯特拉算法、A*搜索算法、距离图、泛洪填充和领地划分。
- **前瞻搜索：**基于前向模型进行搜索，包括模拟推演、束搜索、极小极大算法、期望极大算法和蒙特卡洛树搜索。
- **组合优化：**指派与匹配、动态规划，以及背包式分配。
- **推断与随机化：**对隐藏信息进行信念跟踪、对手建模，以及混合或随机化选择。

我们分析每位冠军的策略目录中的源文件，排除字节完全相同的SDK文件，以及匹配版本、备份、调试、测试或第三方供应商过滤条件的路径。我们根据每个函数的结构和标识符为其分配算法族。对于超过40行的函数，我们以20行为窗口分别进行分类。我们将输入/输出、解析和协议代码视为基础设施并予以排除。某个算法族的占比为其在剩余非注释行中所占的比例。表10 报告了每个模型–游戏组合的结果，以及跨模型和游戏的等权均值。

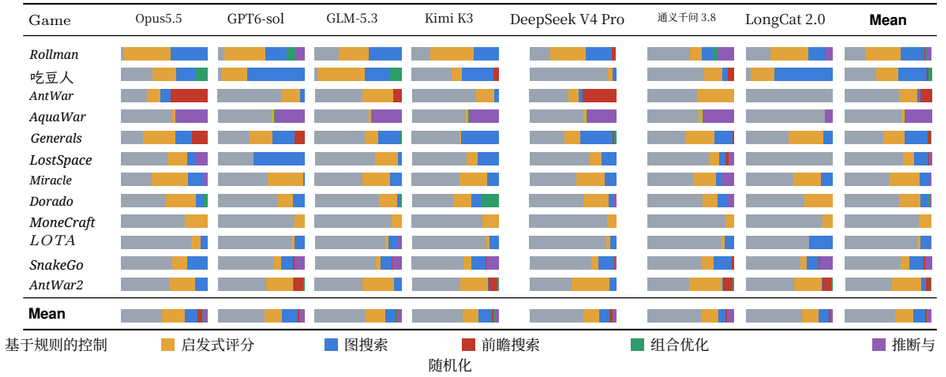

表 10 冠军策略中的算法族。每个条形按算法族划分一个保留的冠军的决策代码，并按非注释代码行数加权。我们排除了基础设施（输入/输出、解析、协议）。行均值和列均值均对各策略赋予相同权重。

保留的冠军主要依赖规则和启发式评分。对 84 个冠军各自的占比进行等权平均后，这两个算法族合计占 79%，图搜索占 13%，其余三个算法族占 8%。84 个冠军中，仅有 7 个的前瞻搜索占比超过 10%，且均来自 *AntWar*、*AntWar2* 和 *Generals*。这些针对最终结果的分类描述了策略组成，但无法确定开发过程中哪些代码变更带来了改进。

不同游戏的算法占比有所不同。图搜索在吃豆人中占 33%，在 *Rollman* 中占 28%；推断在 *AquaWar* 中占 30%。*MoneCraft* 和 *LOTA* 几乎完全由规则构成。在各模型中，Opus5.5 的规则占比最低（47%），评分占比（27%）和前瞻搜索占比（5%）最高；GPT6-sol 的图搜索占比最高（18%）。其他模型的规则占比介于 58% 至 65% 之间。

在每个游戏中，我们计算七种配置的非规则占比（1– 基于规则的占比）与埃洛等级分之间的相关性，对并列值采用平均秩。12 个游戏的斯皮尔曼相关系数均值为 0.35，不同

games. This association does not establish that replacing rules improves a policy.

The classification is automatic and approximate. Identifiers and comments influence it; we assign a mixed function to its dominant mechanism. Models retaining the same starter code have similar shares, as on $AquaWar$ and $LOTA$.

游戏之间存在差异。这种关联并不能证明替换规则会改善策略。

该分类是自动进行的，结果仅为近似。标识符和注释会影响分类结果；我们将混合函数归入其主导机制所属的类别。保留相同初始代码的模型具有相近的占比，$AquaWar$ 和$LOTA$中的情况便是如此。

26